



Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
AUT Tanszék

SZAKDOLGOZAT FELADAT
Szűcs Tamás Dominik
Üzemmérnök-informatikus hallgató részére

**IoT-alapú ipari adatgyűjtési, adatmegjelenítési és adatfeldolgozási rendszer
fejlesztése**

Az Ipar 4.0, vagy ipari digitalizáció rengeteg új technológiai eszközt biztosít a gyárak számára, ami lehetővé teszi a gépek közötti kommunikációt, intelligens rendszerek kialakítást, az automatizált döntéshozatalt, ezek alkalmazásával okos gyárak felépítését. Célja elsősorban a folyamatok hatékonyságának maximalizálása, a rugalmasság megteremtése a gyártásban, a megbízhatóság növelése. Egyszerű, IoT alapú megoldások alkalmazásával a hagyományos gépek, berendezések is felokosíthatók, így alacsony költséggel, ma már a kisvállalatok is el tudnak indulni a digitalizálással.

A szakdolgozat keretében Hallgató feladata, egy olyan IoT alapú rendszer tervezése és implementálása, amely képes ipari környezetben adatokat gyűjteni klasszikus szerszámgépek működéséről okos érzékelők felhasználásával, melyek a mért adatokat egy központi adatbázisba továbbítják. Az adatok valós időben megjeleníthetők, az összegyűjtött adatokon különböző elemzéseket lehet végezni.

A hallgató feladatának a következőkre kell kiterjednie:

- Ismerje meg az ipari adatgyűjtő rendszerek felépítését, tekintse át a gyűjthető adatok körét, az alkalmazható szenzorokat és mikrovezérlőket, mérési módszereket;
- Vizsgálja meg és hasonlítsa össze a gyűjtött adatok feldolgozásának különböző lehetőségeit;
- Tervezzon egy mérőrendszert egy hagyományos szerszámgép (pl. eszterga) üzemi adatainak gyűjtéséhez, feldolgozásához és vizualizálásához. A feladat magába foglalja az elektronika, a mechanikus alkatrészek (szenzorok rögzítése, dobozolás stb.), valamint a kommunikáció és a szoftver rendszer megtervezését;
- Építse meg a rendszert, szerelje fel a szenzorokat a kiválasztott gépre, készítse el a mikrokontrollerek programjait és a központi adatfeldolgozó alkalmazást;
- Analizálja a gyűjtött adathalmazt, mérje fel az elemzési lehetőségeket és tegyen javaslatot az adatok gyakorlati felhasználására (pl. megelőző karbantartás);
- Tesztelje az alkalmazást, tegyen javaslatot az ipari alkalmazásra.

Tanszéki konzulens: Kovács László, ipar 4.0 témavezető

Budapest, 2026. március 20.

Dr. Charaf Hassan
egyetemi tanár
tanszékvezető



Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
AUT Tanszék

Szűcs Tamás Dominik

**IOT-ALAPÚ IPARI
ADATGYŰJTÉSI,
ADATMEGJELENÍTÉSI ÉS
ADATFELDOLGOZÁSI
RENDSZER FEJLESZTÉSE**

KONZULENS

Kovács László

BUDAPEST, 2026

Tartalomjegyzék

1. Bevezetés	9
1.1. Az adat fontossága a döntéshozatalban.....	9
1.1.1. Fejlődéstörténet.....	9
1.1.2. Az adat relevanciája.....	11
1.1.3. Szenzorok és mérendő paraméterek	13
1.1.4. Adattípus és adattárolás	15
1.2. IoT bevezetése	16
1.2.1. IoT hardver eszközök.....	16
1.2.2. Mikrovezérlők.....	17
1.2.3. Single Board Computer	18
1.2.4. PLC	19
1.2.5. Kommunikáció.....	20
1.2.6. Mesterséges intelligencia integrálása.....	20
1.3. Felhőalapú számítástechnika	21
1.3.1. Népszerű IoT platformok.....	22
1.3.2. Saját platform bevezetése	24
2. Kiinduló állapot és továbbfejlesztési terület.....	26
2.1. Architektúrális felépítés	26
3. Mérőegység felépítése	28
3.1. Hardverelemek szerepei.....	28
3.1.1. Mikrovezérlők.....	28
3.1.2. Szenzorok és egyedi mérőkészülékek.....	28
3.1.3. Tápellátás	33
3.1.4. Nyomtatott áramkör és ház	34
3.2. ESP32 firmwarek	34
3.2.1. ESP1 taszkjai	34
3.2.2. ESP2 taszkjai	35
3.3. Adatgyűjtés	36
3.3.1. Motorhőmérséklet mérése.....	36
3.3.2. Háromfázisú motor áramfelvételének mérése	36
3.3.3. Főorsó fordulatszám mérése	37

3.3.4. Főorsó rezgésének mérése	37
3.4. Karbantartási felület.....	38
3.4.1. A menürendszer működése	38
3.4.2. Kirajzolás	39
3.5. Kommunikáció a rendszerben	41
3.5.1. ESP-k közötti kommunikáció	41
3.5.2. Mérésadatok küldése.....	43
3.6. Hiba és állapotdetekció	44
4. Szenzoradatok tárolás	45
4.1. MQTT bróker.....	45
4.1.1. Bróker konfigurálása.....	45
4.1.2. MQTT Topic felépítés	46
4.2. Adattárolás	47
4.2.1. Influxdb dockeres szolgáltatásfelépítése	47
4.3. Szerveroldali ágens - Telegraf	48
4.3.1. Pluginok	48
4.3.2. Ágens előzetes konfigurációja	50
5. Adatelemzés és utófeldolgozás	51
5.1. Backend modul felépítése	51
5.1.1. Adatfeldolgozás	51
5.1.2. Elemzési módszerek és algoritmusok	52
5.1.3. Utófeldolgozás rögzítése.....	53
6. Sztochasztikus szimulációból komplex vezérlés	54
6.1. Korai fázis.....	54
6.2. Fokozatok bevezetése	54
6.3. Kopásszimuláció	55
6.4. Végső megoldás	55
7. Felhasználói felület	56
7.1. Plotly Dash.....	56
7.1.1. Adatelérés	56
7.1.2. Dashboard főképernyő és aloldalak.....	57
8. Prediktív karbantartási funkciók.....	63
9. Tesztelés	65
10. Elért eredmények, tapasztalat	67

11. Továbbfejlesztési javaslat.....	68
11.1.1. Kisebb mértékű fejlesztések	68
11.1.2. Nagyobb mértékű fejlesztések	68
12. Irodalomjegyzék.....	69
13. Függelék.....	70
13.1. I. Függelék Nyilatkozat generatív mesterséges intelligencia alkalmazásáról.	70

HALLGATÓI NYILATKOZAT

Alulírott **Szűcs Tamás Dominik**, szigorló hallgató kijelentem, hogy ezt a szakdolgozatot meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem. A Függelékben egyértelműen megjelöltem, hogy a mesterséges intelligencia eszközeit alkalmaztam-e a dolgozat elkészítéséhez; amennyiben igen, annak módját és mértékét a táblázatban közöltem. Tudomásul veszem, hogy a mesterséges intelligenciával generált tartalomért - annak mértékétől függetlenül - teljes felelősséggel tartozom.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző(k), cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK az interneten nyilvánosan hozzáférhetővé tegye, a munka teljes szövege pedig a BME Címtárban regisztrált személyek számára elérhető legyen. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Kelt: Budapest, 2026. 06. 07.

.....
Szűcs Tamás Dominik

Összefoglaló

A negyedik ipari forradalom meghatározó mérföldkő a modern tömeg és egyedi gyártásban. Az internet és a technológiai fejlődése megteremti az emberek, gépek és vállalatok számára az optimalizált és hatékony termék-előállítás lehetőségét. Ebben a célban meghatározó erő az adat, amit felhasználva elemzéseket készíthetünk a működésről és következtetéseket vonhatunk le, amivel a jövő eseményeire is fel tudunk készülni. Ennek a legelterjedtebb módja az ipari szenzorok és feldolgozó egységek telepítése, amik együttvéve átfogóbb méréseket lehet végezni, analitikai módszerekkel mintákat felismerni és akár meghibásodást előre jelezni. Ezzel ütemezni lehet a karbantartást és akár előzetes problémafeltárást is lehet végezni, megelőzve a gyártásleállás valószínűségét.

Jelenlegi scenárióban egy klasszikus esztergagép a tárgyalt munkaeszköz, aminek kulcsfontosságú tényezői (fordulatszám, motorhőmérséklet, rezgés és áramfelvétel) megfigyelésére modern szenzorok állnak rendelkezésre, valamint mikrokontroller alapú vezérlés. Minimális előfeldolgozás után egy távoli szerveregységre MQTT protokoll segítségével kerül az adatmennyiség. Ott egy többkomponensű pipeline-ba kerül, aminek első lépése a nyers adatok idősoros adatbázisba mentése. Mennyiségek további feldolgozása után az adatok valós időben egy műszerfal segítségével a gépkezelő felügyelheti. Emellett előzetesen betanult modellek alapján képes az eszközök állapotát felismerni és javaslatot tenni a karbantartásra.

A projekt célja egy meglévő termék továbbfejlesztése, ami megfizethető megoldást nyújt régebbi szenzormentes esztergagépek felokosítására, így bevezetve az Ipar 4.0 világába a munkagépet, mégpedig minimális változtatással az eszközben nagyobb átépítés nélkül.

Abstract

The Fourth Industrial Revolution is a defining milestone in modern mass and custom manufacturing. The internet and technological advancements are creating opportunities for people, machines, and companies to achieve optimized and efficient production. Data is a key factor in this endeavor; by utilizing it, we can analyze operations and draw conclusions that help us prepare for future events. The most common way to achieve this is by installing industrial sensors and processing units which when combined, allow for more comprehensive measurements, identification of patterns using analytical methods and even the prediction of failures. This allows for scheduled maintenance and even early problem detection thereby reducing the likelihood of production downtime.

In this scenario a classic lathe is the workpiece and modern sensors are available to monitor its key factors (RPM, motor temperature, vibration and power consumption), along with microcontroller based control. After minimal preprocessing the data is transmitted to a remote server via the MQTT protocol. There it enters a multi component pipeline, for the first step is saving the raw data to a timeseries database. After further processing of various aspects, the machine operator can monitor the data in real time via a dashboard. Additionally based on pre-trained models the system is capable of detecting equipment wear and recommending maintenance.

The goal of the project is to further develop an existing product that offers an affordable solution for upgrading older sensorless lathes bringing the machine into the world of Industry 4.0 with minimal modifications to the equipment and without major overhauls.

1. Bevezetés

1.1. Az adat fontossága a döntéshozatalban

A termékellenőrzés megkerülhetetlen szempont vállalatok számára a versenytársakkal szemben is, valamint termék a megfelelésének a modern kor elvárásainak. Ezen kulcsinformációk tudatában pontosan behatárolhatók lettek a fejlesztési és üzleti döntések iránya.

A vállalkozások visszajelzéseket kaphatnak az ügyfelektől a meglévő termékeikről és szolgáltatásaikról. Ezeket a visszajelzéseket felhasználhatják a fejlesztésekre, problémák megoldására, valamint olyan új ajánlatok kidolgozására, amelyek megfelelnek az ügyfelek igényeinek. [1]

Alkalmazható még a belső működés, az ellátási lánc menedzsmentje és a gyártási folyamatok területén. A működési adatok elemzése hatékonyságnöveléshez, költségcsökkentéshez és a munkafolyamatok egyszerűsítéséhez vezethet, ami végső soron javítja a vállalkozás általános teljesítményét. [1]

Az adatgyűjtés képi az alapját a prediktív elemzésnek, amelynek során a múltbeli adatok felhasználásával előre jelezzük a jövőbeli trendeket és eredményeket. Ez a képesség segít a vállalkozásoknak abban, hogy proaktív döntéseket hozzanak, ahelyett hogy csak az események bekövetkezte után reagálnának. [1]

1.1.1. Fejlődéstörténet

Ipar 1.0 - Gépesítés

Megjelenése kb. 1740 és 1850 között először az Egyesült Királyságban alakult ki, legnagyobb hatása főleg az agráriparban mutatkozott meg. A gépesítés miatt kevesebb földművelés több munkát tudott elvégezni, így egyre több embert tudott ellátni élelemmel. Logisztikai szinten, a gőzhajó az addigi vitorláhajókhoz képest nagy előrelépést jelentett, mivel a széljárás kiszámíthatatlanságához képest sokkal megbízhatóbb. Így lehetővé vált az interkontinentális szállítás. A vidék népessége a városokba áramlott, táplálva ezzel a fellendülő ipar munkaerő szükségletét. Az üvegipar, a bányászat és a textilipar is nagyban profitált ebből, mivel a gépesítés bevezetésének köszönhetően nagyobb mennyiségű terméket tudtak előállítani.[2]

Ipar 2.0 - Elektromos áram megjelenése

A második ipari forradalom főként Nagy-Britanniában, Amerikában és Németországban indult az 1870-es években és „technológiai forradalom” néven vált ismertté. Ebben az időszakban vezették be az elektromos technológiát, amely eddig soha nem látott módon hajtotta meg a gépeket. Ezáltal a gépek sokkal hatékonyabbá váltak. Ekkor vált valósággá a tömegtermelés, ami a futószalag bevezetésének köszönhető. Ez tette lehetővé az autók és repülőgépek feltalálását és sikereit. [2]

Ipar 3.0 - Számítógépek bevezetése

Az 1970-es években bekövetkezett harmadik ipari forradalom elindította az informatika és a számítástechnika korszakát, amelyet „digitális forradalomnak” neveznek. Bár ezek a számítógépek jóval nagyobbak voltak, mint a maiak, mégis megalapozták a mai számítógépek fejlődését. Ezek a gépek nagyrészt automatizáltak voltak, mégis szükségük volt emberi irányítóra. A gyártás és az automatizálás az internet hozzáférés, a hálózati összeköttetés és a megújuló energiaforrások révén is fejlődött. [2]

Ipar 4.0 - Ember, technológia és fenntarthatóság

Bár vitatott, hogy pontosan mikor kezdődött az Ipar 4.0, a kifejezést 2011-ben alkották meg, amikor az iparágban jelentős növekedés volt tapasztalható az intelligens gépek, tárolórendszerek és olyan gyártóberendezések terén, amelyek nem igényelnek emberi beavatkozást. Megszületett az Internet of Things (IoT), amely a számítógépes érzékelő és megfigyelőképességekkel ellátott gépi eszközök és járművek internetbe kapcsolt hálózata. A remény az, hogy az Ipar 4.0 segítségével több zöld technológia kifejlesztésével a fenntarthatóság három pillére (környezeti, gazdasági és társadalmi) jelentős javulást fog tapasztalni. [2]

Ipar 5.0 - A jövő ipara

Ez az időszak lesz, amikor az emberek és a mesterséges intelligencia egymás mellett dolgoznak majd a gyártás hatékonyságának javítása érdekében. Az Ipar 5.0 célja, hogy az emberek és a robotok együttműködésével jobban automatizálják a gyártási folyamatot és így jobb, személyre szabott megoldásokat nyújtsanak az ügyfeleknek. [2]

Az Internet of Things (IoT) valódi forradalmat jelentett és a big data és az IoT több szempontból is elválaszthatatlanul összefonódik. Számtalan IoT eszközt gyűjt jelenleg adatokat, akár valós időben, akár egyszeri felhasználásra.

Ezeknek az adatoknak a mennyisége és áramlási sebessége olyan nagy, hogy azok hasznosíthatóságához big data feldolgozási kapacitásokra van szükség. A big data segíti még a technológiai vállalkozásokat az IoT eszközökben rejlő analitikai képességek kezelésében és kihasználásában. Az analitikai folyamatok fejlődése és azok integrálása az IoT-eszközökbe mindkét technológiát a mai világban elengedhetetlenül fontossá teszi.[3]

1.1.2. Az adat relevanciája

Az adatok fontos erőforrásnak bizonyulnak, mivel felhasználhatók az innováció ösztönzésére és hibák javítására. Egyes szervezetek és cégek az adatokat a mesterséges intelligencia vagy a gépi tanulás alkalmazásával újszerű megoldásokat találhatnak a problémákra és elősegíthetik az innovációt a saját szakterületükön.

Az ügyféladatok kiaknázásával a szervezetek személyre szabhatják kínálatukat és az ügyfelekkel való interakcióikat, ami az ügyfélelégedettség és az ügyfélhűség növekedéséhez vezet. Ezen felül az adatok felhasználhatók az ügyfélviselkedés mélyebb megértéséhez, ami lehetővé teszi a szervezetek számára, hogy új növekedési és bevételi lehetőségeket azonosítsanak.

Az adatok értékes erőforrást jelentenek a vállalatok biztonsági intézkedései szempontjából, mivel segítenek a szervezeteknek a biztonsági fenyegetések azonosításában és megelőzésében. Ezen kívül az adatok felhasználhatók olyan prediktív modellek építésére, amelyek a biztonsági incidensek bekövetkezése előtt képesek felismerni azokat a viselkedési mintákat, amelyek biztonsági megsértésre utalnak. Különböző forrásokból származó adatok gyűjtésével és elemzésével a szervezetek azonosíthatják a potenciális biztonsági fenyegetéseket így valós időben reagálhatnak rájuk.

Végül az adatok értékes betekintést nyújtanak a szervezeteknek, ami elősegítheti a jobb döntéshozatalt. A szervezetek betekintést nyerhetnek működésükbe, ügyfeleikbe és piacaikba, hogy adatokon alapuló döntéseket hozzanak, amelyek javítják általános hatékonyságukat és eredményességüket.[4]

1.1.2.1. Karbantartás hatása a termelékenységére

Az ipari berendezések meghibásodása nemcsak termeléskiesést okoz, hanem az ügyfélszolgálat késedelmét, biztonsági problémákat és környezetvédelmi aggályokat is eredményezhet. Tehát látszik, hogy a karbantartás fontossága milyen kiemelt szerepet játszik egy szervezet gyártási folyamataiban.

Az árakra és a megbízhatóságra gyakorolt jelentős hatása miatt a karbantartás alapvető ipari tevékenység, amely jelentősen befolyásolja a vállalkozás versenyképességét a teljesítmény, a minőség és az alacsony költségek tekintetében. A gépek, berendezések vagy eszközök bármilyen nem tervezett leállása károsíthatja vagy megszakíthatja a vállalkozás alapvető működését, ami esetleg súlyos bírságokhoz és a vállalat hírnevének visszafordíthatatlan károsodásához vezethet. Például „2013-ban az Amazon 424.438 ezer dollár értékű árbevételt veszített 49 perces leállása miatt.” (A kutatást a kollektív oldal közölte a Ponemon Institute adataira hivatkozva, miszerint 1 percnyi leállás 8.600 dollárnyi veszteséget okoz) [5]

Az adatelemzés és az adatalapú modellek terén bekövetkezett technológiai fejlődés elősegíti az iparágak átállását a hagyományos megelőző karbantartásról (**PM**) a prediktív karbantartásra (**PdM**). A modern gyártórendszerek hatalmas mennyiségű érzékelőadatot generálnak, ami lehetővé teszi a berendezések folyamatos megfigyelését. A **PdM** valós idejű adatokat használ a problémák előrejelzésére és a karbantartás optimalizálására, ezzel csökkentve a költségeket és az állásidőt, ellentétben a **PM**-mel, amely ütemezett feladatokra támaszkodik. Az adatközpontú algoritmusok előre jelzik a berendezés fennmaradó élettartamát (**RUL**) és bonyolult adatminták elemzésével azonosítják a rendellenességeket. Az eszközközelés forradalmasul ezzel az adatközpontú stratégiával, amely javítja a működési teljesítményt, a hatékonyságot és a megbízhatóságot.

A pontos prediktív karbantartási eredményekhez az adatalapú modelleknek a tudás és fizikaalapú modellekkel való integrálására van szükség. A hatalmas mennyiségű korábbi adat és a gépi tanulási technikák felhasználásával az adatalapú modellek kiválóan alkalmasak a trendek felismerésére és a meghibásodások előrejelzésére. Azonban az elégtelen adatokkal rendelkező vagy új meghibásodási módokkal járó forgatókönyvek kihívást jelenthetnek a rendszereknek. A kopás és a meghibásodás alapjául szolgáló mechanizmusokba való betekintést nyújtva a valós modellek fizikai viselkedésének alapelvein alapulnak.

A tudásalapú modellek a valós tapasztalatok és a kontextusismeret ötvöztetésével pótolhatják a hiányosságokat, mivel szakértői szakterületi ismeretekből és megállapított kritériumokból származnak. A prediktív karbantartási rendszerek kihasználhatják ezen megközelítések erősségeit azok kombinálásával: tudásalapú modellek kontextuális relevanciáját, adatalapú modellek alkalmazkodóképességét és skálázhatóságát, valamint fizikaalapú modellek alapvető pontosságát. Ez végül robusztusabb és megbízhatóbb előrejelzésekhez vezet.

Az Ipar 4.0 keretében a prognosztika egy gép fennmaradó élettartamának (**RUL**) becslését jelenti a gép vagy alkatrészeinek várható kopási állapotának előrejelzése révén. Az előrejelzést a modern ipar karbantartás területén kulcsfontosságú elemnek tekintik, mivel lehetővé teszi a **RUL**-t meghatározó modellelőrejelzések alapján mind rövid, mind hosszú távú intézkedések ütemezését. Ebben az összefüggésben a szakirodalom számos módszertant tárgyal, beleértve a tudásalapú, a fizikaalapú és az adatközpontú modelleket. A bonyolultabb helyzetek kezelése érdekében a tudásalapú és a fizikaalapú modelleket gyakran integrálják az adatközpontú modellekkel, amelyek statisztikai vagy gépi tanulási technikákat alkalmazhatnak.

Az anomáliadetekciós módszerek közül az adatközpontú megközelítéseket két kategóriába sorolhatjuk: a statisztikai és a gépi tanulási módszerekre. A szakirodalomban számos statisztikai modell és módszer szerepel a prediktív karbantartás (**PdM**) elemzésére, ideértve a rejtett Markov-modellek (**HMM**), Wiener-folyamat modellek (**WPM**), gamma-folyamat modellek, arányos kockázati modellek és autoregresszív mozgóátlag (**ARMA**) modellek. A gépi tanulási megközelítések többek között magukban foglalják az **ANN**-t és annak változatát, a támogató vektoros gépeket (**SVM**), a random forest (**RF**), az **xGBoost**-ot és az önszerveződő térképeket (**SOM**).[6]

1.1.3. Szenzorok és mérendő paraméterek

Az ipari és kereskedelmi alkalmazásokban használt forgógépek több mint 90%-a gördülőcsapágyakat használ. A motor meghibásodott alkatrészeinek eloszlása jól láthatóan azt mutatja, hogy a **PdM**-érzékelő kiválasztásakor fontos a csapágyfigyelésre összpontosítani. A lehetséges meghibásodások észlelése, diagnosztizálása és előrejelzése érdekében a rezgésérzékelőnek alacsony zajszinttel és széles sáv szélességgel kell rendelkeznie a pontos következtetéshez.

Az eszközön alkalmazott prediktív karbantartási érzékelő teljesítménye összefügg azzal, hogy mennyire fontos az eszköz folyamatos és megbízható működése a teljes folyamatban és nem az eszköz költségeinek rovására.

A motor rezgésében vagy mozgásában rejlő energia mennyisége (csúcsérték, csúcs-csúcs érték és effektív érték) alapján többek között megállapítható, hogy a gép kiegyensúlyozatlan-e vagy rosszul van-e beállítva. Egyes hibák például a csapágy vagy áttétek, nem annyira nyilvánvalóak különösen a korai szakaszban és nem azonosíthatók vagy jelezhetők pusztán a rezgés növekedése alapján. Ezek a hibák általában nagy teljesítményű, prediktív karbantartási rezgésérzékelőt igényelnek, alacsony zajszinttel ($<100 \mu\text{g}/\sqrt{\text{Hz}}$) és széles sávzélességgel ($>5 \text{ kHz}$), amelyhez nagy teljesítményű jellánc, feldolgozás, adó-vevők és utómunkálatok társulnak.

A gyorsulásmérők a leggyakrabban használt rezgésérzékelők, a rezgéselemzés pedig a legelterjedtebb állapotalapú karbantartási (**PdM**) technika, amelyet főként nagy forgógépeken, például turbinákon, szivattyúkon, motorokon és hajtóműveken alkalmaznak.

A sebességérzékelők alkalmasak tárgyak sebességének mérésére (általában kerekek, motorok vagy forgó részecskék esetében). A vállalatok sebességmérőket, gyorsulásmérőket, LIDAR-érzékelőket, fordulatszám-mérőket, Doppler-radarokat stb. használnak. Elérhetők még infravörös (IR) érzékelők a fény felhasználásával, amik optikai elven alapulnak. Egyik fajtája a visszaverő IR-érzékelők adója és detektora egymás mellett helyezkedik el, az objektum felé irányítva. Másik az átmenő típusú érzékelők, amik LED-eket és fotodiódákat használnak a közöttük elhaladó objektum észlelésére.

Egy másik érzékelőcsoport a hangérzékelésre specializálódott érzékelők, amelyek általában mikrofonok segítségével működnek. Amikor egy hangérzékelő jelet észlel, a feszültség-szintet a megfelelő hangszintre konvertálja. A jelszinteket pedig utófeldolgozás után könnyű analizálni.

A nyomásérzékelők azonosítják a mérési tárgyakban vagy a környezetben fellépő nyomáseltéréseket. A változás észlelése általában barometrikus, piezoelektromos, kapacitív, optikai vagy rezonáns érzékelési elveken alapul. Ilyen típusú érzékelők például a Bourdon-csővek, a membránok vagy nyomásmérők.

A hőmérsékletérzékelők általában a gépek állapotának változásait vagy a gyárban fennálló kritikus helyzeteket észlelik (különösen veszélyes környezetben). Az érzékelők a hőmérsékleti adatokat közvetlenül (ellenállásos hőmérsékletérzékelők, termisztorok és hőelemek) vagy közvetve (infravörös érzékelők) mérik.

Az energia és árammérő érzékelők biztosítják az elektromos kábelek szigetelésének megfelelőségét, eszköz energiafelhasználás mérését és esetekben áramfelvétel csökkentését. Jelenleg optikai szálakat és a mágneses folyadékokat használják széles körben alkalmazási sokoldalúságuknak köszönhetően. Az elektronikus érzékelők esetében ezek lehetnek például gázérzékelők, amelyek rugalmasságuknak, az intelligens elektronikával való integrálhatóságuknak, valamint az energiafogyasztással és az elektronikus alkatrészek teljesítményével kapcsolatos mechanikai ellenállásuknak köszönhetően képesek valós időben érzékeny elemzéseket végezni.[7]

1.1.4. Adattípus és adattárolás

Hosszú ideig az érzékelőadatok analóg jellegűek és pillanatnyi értékeket mutattak. Ma már ezek nagy része, különösen ipari környezetben digitális módon kerül rögzítésre. A legtöbb érzékelő esetében ehhez analóg-digitális átalakítóra (ADC) van szükség, amelyet általában magába az érzékelőbe építenek be. Miután az adatok digitális formátumba kerültek szükség szerint továbbíthatók, tárolhatók és elemezhetők. Az információk egyik legfontosabb felhasználási területe a trendelemzés lehetővé tétele mellett a valós idejű adatvizualizáció.

Az összefüggések megállapításában a strukturált adatok nyújtanak a legnagyobb segítséget, ezen belül is az idősorokba rendezettek. Az idősoros adatok alapvetően egy adatsoroból álló sorozatot jelentenek, amelyek mindegyike egy adott időponthoz kapcsolódik. Az egyes adatpontok időbeli változásának megértése számos statisztikai és üzleti elemzés alapját képezi.

Valahányszor egy hálózaton belül művelet történik, naplófájlok jönnek létre, amelyek lehetővé teszik a felhasználók számára, hogy nyomon kövessék az egyes tevékenységeket azok bekövetkezte után. [8]

1.2. IoT bevezetése

A dolgok internete (IoT) olyan fizikai eszközök, járművek, háztartási gépek és egyéb tárgyak hálózatát jelenti, amikbe hálózati kapcsolódási lehetőségek vannak beépítve, lehetővé téve számukra az adatok gyűjtését és megosztását.

Az IoT alkalmazási lehetőségei hatalmasok és sokrétűek, hatása pedig már most érezhető számos iparágban, többek között gyártásban, közlekedésben, egészségügyben és mezőgazdaságban. Vállalati környezetben az IoT eszközöket számos mennyiség például hőmérséklet, páratartalom, levegőminőség, energiafogyasztás és gépek teljesítményének figyelemmel követésére használják. Ezeket az adatokat valós időben analizálva azonosíthatók olyan mintázatok, tendenciák és rendellenességek, amelyek segítségével a vállalkozások optimalizálhatják működésüket és javíthatják termelékenységüket.

1.2.1. IoT hardver eszközök

Számos gyártó szolgáltat univerzális platformot vagy bizonyos eszközcsaládhoz célzott IoT felhasználásra. Összességében szakmailag jól dokumentált leírások társulnak hozzájuk, illetve felhasználók által készített könyvtárak, gyakori problémákra megoldást szolgáló kódrészek és hasonló projektek is.

A platformfejlesztők szinte évente hoznak új eszközt a piacra, de egyes verziók közkedveltsége miatt időtállóan megtalálható még. Így legtöbbször a felhasználási cél és a hozzá kapcsolható komponensek határozzák meg, milyen eszköz is kerül kiválasztásra. Szempont lehet a fizikai processzormag, áramfelvétel, kommunikációs technológiák, amiből talán a legérzékenyebb az áramfelvétel kérdése, ha nem telepről üzemelne, hanem akkumulátorról. Ilyenkor előzetes kalkuláció segíthet az üzemidő meghatározásában.

1.2.2. Mikrovezérlők

Ipari rendszerekben elterjedtek az Espressif által kínált mikrovezérlők, amik 3.3 [V] tápfeszültséget igényelnek. Az alábbi részben összefoglaltam szériák alapján és felhasználási területeik szerint a vezérlőket:

- ESP8266 régebbi típusú mikrovezérlő 2014 óta elérhető, ideális kezdő szintű IoT-alkalmazásokhoz, például alapvető érzékelőrendszerekhez, egyszerű otthoni automatizáláshoz. Kedvező ára és alacsony energiafogyasztás miatt máig előszeretettel vásárolják. Processzora 32bit RISC architektúrájú 80-160MHz órajellel, 64KB utasítás és 64KB adat memóriával rendelkezik.
- Utódja az ESP32, ami két processzormaggal és 240MHz-es órajellel, 520KB osztott memóriával, Bluetooth kommunikációval, több interfésszel és DAC/ADC-vel lett felszerelve. Architektúra ettől a modelltől kezdve RISC-V típusra váltott.
- ESP32-H széria az alacsony fogyasztású működésről és minél hosszabb akkumulátoros üzemidő megteremtésére készült, így alkalmazható fitnessórának és HMI interfésznek is.
- ESP32-E széria a magasabb teljesítményi fejlesztőeszköz ideális hubok és eszközök vezérlésére, vezeték nélküli videó streamelésre és akár AR/VR eszközökhöz is.
- ESP32-P célja a fejlett ember-gép interfészek, a hatékony peremhálózati feldolgozás, valamint a megnövekedett I/O kapcsolati igények hatékony támogatása.

Az Espressif-hez hasonlóan az Arduino is gyárt mikrovezérlőket IoT alkalmazásokhoz. Az ESP32 egy nagy teljesítményű, kétmagos, WiFi/Bluetooth-kompatibilis IoT-chip, míg az Arduino egy kezdőknek is könnyen kezelhető fejlesztői platform. Az ESP32 jelentősen jobb teljesítményt (240MHz vs 16MHz), több memóriát és beépített hálózati kapcsolódási lehetőségeket kínál.

Arduinohoz az MKR széria különböző modelljei és a hozzájuk megvásárolható kiegészítő modulok egy jó kiindulási alapot biztosítanak a kezdő, vagy akár haladó fejlesztők számára. Elérhető WiFi, vagy akár LoRa képes változat is, ami két gyakori szabvány ipari környezetben.

A mikrovezérlőkön túl mindkét cég biztosít egy kompatibilis felhőalapú szolgáltatást is, az Espressif esetében ez az ESP RainMake egy rendkívül rugalmasan testreszabható IoT platform, amely eszközfirmware-t, mobilalkalmazásokat, felhőalapú háttérrendszert, hangsegéd-integrációkat és eszközközkezelő irányítópultot kínál. Az Arduino oldaláról elérhető az Arduino Cloud ami, egy felhasználóbarát felhőalapú platform kifejezetten az Internet of Things (IoT) projektek fejlesztésére tervezve. A kezdők számára közzéadott funkciók és projektek bőven elérhetőek például távoli világításvezérlő, környezeti megfigyelő vagy otthoni eszközközkezelő rendszereket hozzanak létre mindezt anélkül, hogy saját szervert kellene üzemeltetniük vagy bonyolult kódot kellene írniuk.

1.2.3. Single Board Computer

Amennyiben egy adott alkalmazás már meghaladja a mikrovezérlők számítási kapacitását vagy funkcionalitását, az egykártyás számítógépek (**SBC-k**) hatékony alternatívát kínálnak. Bár méretük kompakt, teljesítményük nagyságrendekkel felülmúlhatja a hagyományos mikrovezérlőkéét. Energiafogyasztásuk ezért jellemzően magasabb, így tervezéskor ezt is figyelembe kell venni. Az **SBC-k** többsége teljes értékű operációs rendszert futtat leggyakrabban valamilyen Linux disztribúciót, de egyre gyakoribb az Android vagy akár specializált konténer alapú rendszerek használata is.

Az egyik legismertebb termékcsalád a **Raspberry Pi 5**, amely a korábbi generációkhoz képest jelentős előrelépést hozott. A négymagos több mint 2 GHz-es processzora mellett választható 4 vagy 8 GB RAM, valamint a PCIe csatoló megjelenése lehetővé teszi gyors adattárolók vagy egyéb nagy sávszélességű perifériák használatát. A platformot 2025-2026 körül tovább erősítették optimalizált operációs rendszerekkel és jobb AI gyorsítási lehetőségekkel. A mesterséges intelligenciára kihegyezett alkalmazások esetében kiemelkedő szerepet töltenek be az NVIDIA megoldásai például a **NVIDIA Jetson Nano** és az újabb Jetson modellek. Ezek a rendszerek GPU gyorsítással rendelkeznek, így alkalmasak neurális hálók futtatására, képfeldolgozásra vagy beszédfelismerésre, akár valós időben is.

Kedvező ár-teljesítmény arányú alternatívát kínál például az **Orange Pi 5**, amely többmagos processzorral és akár 16 GB memóriával is elérhető. Az utóbbi években számos további gyártó lépett piacra (pl. **Radxa**, **Banana Pi**, **ASUS Tinker Board**), így az **SBC**-k kínálata jelentősen diverzifikálódott. Ezek az eszközök gyakran speciális funkciókkal kerülnek kialakításra például beépített AI gyorsítóval, több hálózati interfésszel vagy ipari csatlakozókkal felszerelvek.

1.2.4. PLC

Az ipari automatizálásban a **PLC**-k (Programozható Logikai Vezérlők) évtizedek óta alapvető szerepet töltenek be. Hagyományosan megbízható, determinisztikus vezérlési feladatokra tervezték őket, így elsősorban gépek és gyártósorok működtetésére használják. Az ipar digitalizációjának előre törésével különösen az Ipar 4.0 és az Internet of Things (IoT) térnyerésével megjelent az igény a **PLC**-k által gyűjtött adatok magasabb szintű feldolgozására. Ennek hatására olyan edge és IoT eszközök jelentek meg, amelyek képesek a **PLC**-k különböző ipari kommunikációs protokolljain (pl. **OPC UA**, **Modbus**, **PROFINET**) keresztül adatokat gyűjteni, előfeldolgozni majd azokat felhőalapú vagy helyi IT rendszerek felé továbbítani.

Erre jó példa a **Siemens SIMATIC IOT2000** család, amely kifejezetten ipari adatgyűjtési és feldolgozási feladatokra lett tervezve. Az ilyen eszközök képesek az üzemi berendezésekből származó adatok aggregálására, lokális elemzésére (edge computing) majd azok biztonságos továbbítására különböző IoT platformok felé. A modern rendszerek már támogatják a konténerizált alkalmazásokat (pl. **Docker**), így rugalmasan telepíthetők egyedi analitikai vagy gépi tanulási megoldások is.

A programok megvalósításához többféle programozási nyelv áll rendelkezésre például Structured Text, Ladder Diagram, Function Block Diagram és Instruction List. Fejlettebb mikrovezérlők terén akár magasabb szintű nyelveket is lehet használni, mint a **Python** vagy a **C/C++**. Az IO-bővítő modulok segítségével további szenzorok, illetve akár mikrovezérlők és egyéb edge eszközök is integrálhatók a rendszerbe.

1.2.5. Kommunikáció

Az IoT eszközök energiafogyasztásának és komplexitásának csökkentése érdekében számos vezeték nélküli technológia áll rendelkezésre, amelyek alapvetően két csoportra oszthatók: rövid és hosszú hatótávú kommunikációs képességű megoldásokra. Rövid hatótáv esetén például helyi hálózatokban előnyös a nagy adatátviteli sebesség és az alacsony késleltetés. Ide tartozik a **Bluetooth** és **BLE**, a **Zigbee**, valamint a **WiFi** (IEEE 802.11 szabványcsalád). Hosszabb hatótáv esetén az **LPWAN** technológiák kerülnek előtérbe, amelyek alacsony energiaigény mellett biztosítanak nagy lefedettséget. Ilyen például a **LoRaWAN**, az **NB-IoT** vagy az **5G** alapú kommunikáció. Ezek különösen hasznosak szétszórta, nagy területen elhelyezkedő eszközök esetén.

A megfelelő technológia kiválasztása az adott alkalmazástól függ: fontos szempont az energiafogyasztás, hatótáv, késleltetés, hálózatba kapcsolt eszközök száma, adatmennyiség, protokollok és kommunikáció gyakorisága. Például szenzorhálózatoknál a **mesh** topológiát támogató **Zigbee** előnyös lehet, míg nagy távolságok áthidalására a **LoRa** alapú megoldások alkalmasabbak.

A fizikai kommunikáció mellett az alkalmazásréteg protokolljai is kulcsszerepet játszanak. Az IoT rendszerekben jellemzően könnyű, kis erőforrásigényű protokollokat alkalmaznak, mint például a **REST**, **MQTT**, **CoAP** vagy **AMQP**. Ezek lehetővé teszik a hatékony, skálázható és alacsony késleltetésű adatcserét különböző eszközök és rendszerek között.

1.2.6. Mesterséges intelligencia integrálása

A mesterséges intelligencia és a gépi tanulás mára az IoT rendszerek fontos elemeivé váltak. Segítségükkel nagy mennyiségű szenzoradat elemezhető, ami lehetővé teszi például az anomáliadetektálást, prediktív karbantartást vagy intelligens döntéshozatalt. Felhőalapú feldolgozás azonban jelentős számítási kapacitást és hálózati erőforrásokat igényel, szűk keresztmetszeteket okozhat a rendszerben. Ennek kezelésére egyre nagyobb szerepet kap az **Edge AI**, amely során az adatfeldolgozás részben vagy egészben az eszközökön történik. Erre több alacsony erőforrásigényre optimalizált keretrendszerek felelhetők, mint például a **TensorFlow Lite Micro**, **STM32 X-CUBE-AI** vagy az **uTensor**. Így elérhetővé vált neurális hálók futtatását akár mikrovezérlőkön is.

A hardveroldalon is jelentős fejlődés történt: egyre több mikrokontroller és beágyazott rendszer tartalmaz dedikált AI gyorsítót vagy lebegőpontos egységet.

1.3. Felhőalapú számítástechnika

Az IoT rendszerek tervezése során nemcsak az eszközök kiválasztása, hanem az adatok kezelésére szolgáló infrastruktúra kialakítása is döntő szerepű lehet. A saját (**on premise**) rendszerek kiépítése és üzemeltetése jelentős erőforrást igényel, ezért egyre több vállalat fordul felhőalapú megoldásokhoz, amelyek skálázható, megbízható és könnyen bővíthető környezetet biztosítanak. Felhőalapú megoldások lehetővé teszik, hogy az IoT rendszerek rugalmasan skálázhatók legyenek, miközben csökkentik az üzemeltetési terheket és támogatják a nagy mennyiségű adat hatékony feldolgozását.

A felhőszolgáltatások három fő kategóriába sorolhatók:

- **IaaS (Infrastructure as a Service):**

Az alap szint, ahol a szolgáltató virtuális szervereket, tárolást és hálózati erőforrásokat biztosít. A felhasználó feladata az operációs rendszer és az alkalmazások kezelése. Ilyen szolgáltatások érhetők el például az **Amazon Web Services**, az **Microsoft Azure** vagy az **IBM Cloud** platformokon.

- **PaaS (Platform as a Service):**

Az infrastruktúrán túl fejlesztői környezetet is kínál (operációs rendszer, futtatókörnyezet, eszközök), így a fejlesztők gyorsabban készíthetnek és telepíthetnek alkalmazásokat. Ide tartozik a **Google Cloud** vagy a **Heroku**.

- **SaaS (Software as a Service):**

A legmagasabb szint, ahol kész alkalmazások érhetők el interneten keresztül, telepítés nélkül. A karbantartás és frissítés a szolgáltató feladata. Tipikus példái a **Microsoft 365**, a **Zoom** vagy a **Dropbox**.

1.3.1. Népszerű IoT platformok

Az IoT projektek megvalósításához számos nagyvállalat kínál komplex felhőplatformokat, amelyek lehetővé teszik az eszközök csatlakoztatását, menedzsmentjét és az adatok feldolgozását. Ezek a rendszerek gyakran szorosan integrálódnak az adott szolgáltató egyéb megoldásaival, így teljes ökoszisztémát biztosítanak analitikára, vizualizációra és gépi tanulásra.

1.3.1.1. AWS IoT

Az Amazon Web Services IoT platformja átfogó megoldást kínál az eszközök csatlakoztatására és az adatok kezelésére. Támogatja az olyan protokollokat, mint az MQTT, HTTPS vagy a LoRaWAN és könnyen integrálható más AWS szolgáltatásokkal.

Főbb komponensei közé tartozik például az IoT Core (eszközkapcsolat), az IoT Analytics (adatfeldolgozás), valamint a Kinesis szolgáltatások (streaming). Emellett speciális megoldások is elérhetők például flottakezeléshez vagy videóadatok feldolgozásához. A platform erősen skálázható és elsősorban SaaS jellegű szolgáltatásként működik.

1.3.1.2. Azure IoT

A Microsoft IoT megoldásai több szinten érhetők el. Az Azure IoT Hub egy PaaS alapú szolgáltatás, amely kétirányú kommunikációt biztosít az eszközök és az alkalmazások között, míg az IoT Central egy magasabb szintű gyors fejlesztést támogató (PaaS) platform. A rendszer támogatja az MQTT, AMQP és HTTPS protokollokat és szorosan integrálható más Azure szolgáltatásokkal például az Azure Machine Learninggel vagy a Stream Analytics-szel. Előnye a jó skálázhatóság és a vállalati környezetbe való könnyen illeszthetőség.

1.3.1.3. PTC ThingWorx

A PTC által fejlesztett ThingWorx kifejezetten ipari IoT alkalmazásokra készült. A platform támogatja a berendezések modellezését, valós idejű adatfeldolgozást és prediktív karbantartást. Elérhető PaaS és SaaS formában is és különösen erős a gyors alkalmazásfejlesztésben, valamint a vizualizációban. A modellalapú megközelítés lehetővé teszi komplex rendszerek hatékony felépítését és újrafelhasználását.

1.3.1.4. Siemens Insights Hub

A Siemens által fejlesztett platform (korábban MindSphere) elsősorban az ipari digitalizációt célozza fő irányának. Megteremti az eszközök és gyártórendszerek adatainak gyűjtését, modellezését és elemzését. Támogatja az „asset” alapú modellezést és különböző analitikai valamint vizualizációs eszközöket kínál. Integrálható mind felhőalapú (pl. AWS, Azure), mind helyi rendszerekkel és ipari gatewayeken keresztül csatlakoztatható válik.

1.3.1.5. Google Cloud IoT

A Google IoT megoldásai az utóbbi években átalakultak, de a felhőplatform továbbra is erős adatfeldolgozási és AI képességeket kínál (pl. BigQuery, Vertex AI integráció).

1.3.1.6. IBM Watson IoT Platform

Az IBM megoldása az AI-alapú analitikára és ipari alkalmazásokra fókuszál, különösen prediktív karbantartási és üzleti intelligencia területen.

1.3.1.7. Tulip

Tulip egy felhőalapú no-code/low-code platform, ami főleg a gyártásoptimalizálás és operátori teljesítménynövelési területen lett elkészítve. Webes platformja gyorsan telepíthető megoldások nyújt IoT eszközök vezérlése, megfigyelése és analitikák készítése terén.

Az IoT platformok egyre inkább komplex, integrált ökoszisztémát kínálnak vállalatok számára. A választás során fontos szempont a kompatibilitás, skálázhatóság és az hogy a platform mennyire illeszkedik az adott alkalmazási területhez (pl. ipari, okosváros, járműipar). Az [1.1. táblázat](#) pár pontban összefoglalja az egyes platformok jellemzőit.

PLATFORM	SZOLGÁLTATÁSI MODELL	TÁMOGATOTT PROTOKOLLOK	ERŐSSÉGEK	TIPIKUS FELHASZNÁLÁS
AWS IOT	SaaS	MQTT, HTTPS, LoRaWAN	Nagy skálázhatóság, széles szolgáltatáskör, erős integráció	általános IoT, streaming, edge
AZURE IOT	PaaS / aPaaS	MQTT, AMQP, HTTPS	Vállalati integráció, erős AI/ML támogatás	enterprise IoT
THINGWORX	PaaS / SaaS	MQTT, REST	IIoT fókusz, modellezés, gyors fejlesztés	ipari rendszerek, digital twin
INSIGHTS HUB	SaaS	MQTT, OPC UA	Gyártási rendszerek integrációja, digital twin	ipari analitika
IBM WATSON IOT	SaaS	MQTT, HTTPS	AI alapú analitika, prediktív karbantartás	prediktív rendszerek
GOOGLE CLOUD IOT*	PaaS	MQTT (bridge), HTTPS	Erős adatfeldolgozások (BigQuery), AI integráció	adatvezérelt IoT

1.1 táblázat - IoT platformok összehasonlítása

1.3.2. Saját platform bevezetése

Az IoT platformok fejlesztése összetett feladat, amely folyamatos karbantartást, skálázást és üzemeltetést igényel. A megbízható működés és a skálázhatóság kulcsfontosságú követelmények, amelyek egyedi fejlesztés esetén komoly tervezést és erőforrásokat igényelnek.

Fontos szempont, hogy a saját megoldás nemcsak szoftverfejlesztést jelent, hanem magában foglalja a hardveres infrastruktúra és a hálózat kiépítését is. Ezzel szemben a kész platformok például az **AWS IoT**, **Azure IoT** vagy **Siemens Insights Hub** magas rendelkezésre állást, automatikus frissítéseket és beépített biztonsági megoldásokat kínálnak, így jelentősen csökkentik az üzemeltetési terheket.

Sok esetben ezért előnyösebb egy meglévő szolgáltatás használata különösen akkor, ha fontos a gyors bevezetés stabil működés és a folyamatosan bővülő funkcionalitás. A felhőszolgáltatók emellett rendszeresen integrálnak új technológiákat.

Viszont vannak olyan helyzetek, amikor az egyedi fejlesztés indokolt lehet. Ilyen például, ha speciális követelmények merülnek fel (pl. egyedi kommunikációs protokollok, szigorú adatbiztonsági előírások, szolgáltatómentes működés) vagy ha a rugalmasság és a teljes kontroll elsődleges szempont. Szintén előnyös lehet saját platform kialakítása, ha a rendszernek gyorsan kell alkalmazkodnia új technológiákhoz és a piaci megoldások nem követik folyamatosan ezeket az igényeket. Ezekről a szempontokról egy kisebb összehasonlítás a [1.3.2-es táblázatban](#) lehet olvasni.

SZEMPONT	FELHŐ PLATFORM	SAJÁT FEJLESZTÉS
BEVEZETÉSI IDŐ	Gyors	Lassabb
SKÁLÁZHATÓSÁG	Automatikus	Kézi tervezést igényel
KÖLTSÉG	Használat alapú	Kezdetben magasabb, később optimalizálható
RUGALMASSÁG	Korlátozott	Teljes kontroll
BIZTONSÁG	Beépített, magas szintű	Saját implementáció szükséges
KARBANTARTÁS	Szolgáltató végzi	Saját felelősség
VENDOR LOCK-IN	Magas	Nincs
OFFLINE MŰKÖDÉS	Korlátozott	Megoldható
IPARI INTEGRÁCIÓ	Gyártófüggő	Teljesen testreszabhatóság

1.3.2 táblázat - IoT platformok összehasonlítása

2. Kiinduló állapot és továbbfejlesztési terület

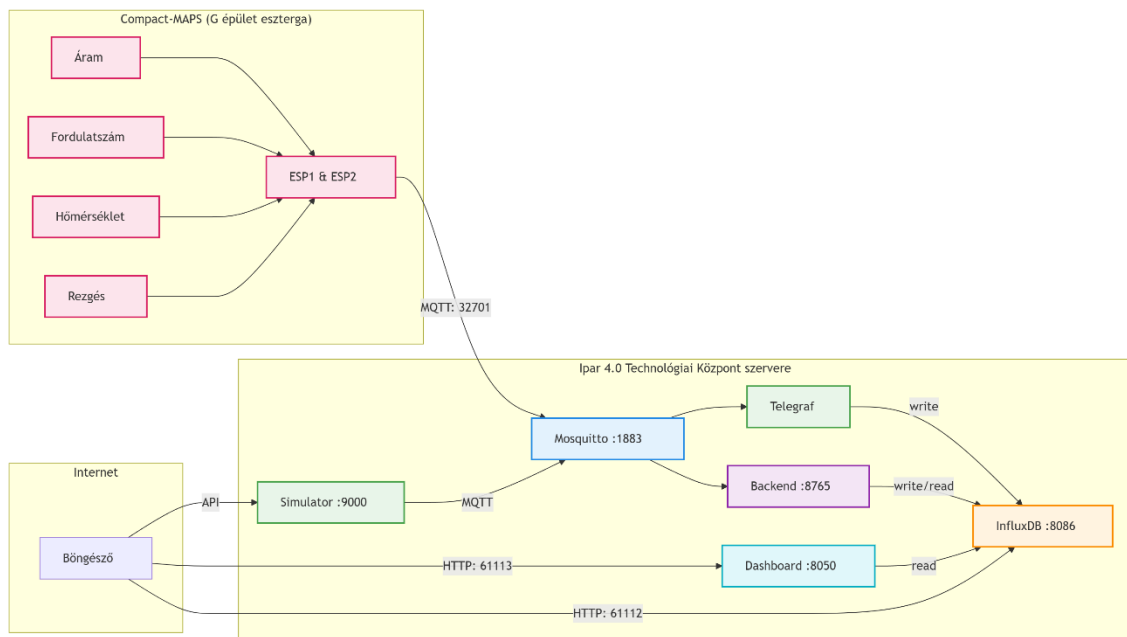
Kezdő állapotnak, egy több éve üzemelő projektet vettem át, ahol a szenzoros modul felőli adatküldés egy bizonyos ideje megszűnt. Szerveroldali szoftvercsomagja azonban kifogástalanul működött. Először laborkörülmények között teszteltem a szenzoros rendszer breadboard-on, ahol gyorsan szembejött a menüvezérelt kijelző működési problémája, ami instabil állapota képösszeeséshez és hibás képernyőfrissítésekhez vezettek. Emellett a szerveroldali demonstrációhoz használható szimulációs funkció egyszerűsége révén inkább csak egy véletlenszám generáló volt a rendszerbe, nem adata vissza a valódi működés érzetét.

A projekt célja, hogy a meglévő IoT adatgyűjtő rendszert tovább fejlesszem, ahol klasszikus szerszámgépek okosíthatók fel szenzoros adatgyűjtési és modern dashboard vizualizációs rendszerekkel. Célja még a gépkezelői felületnek, hogy a mért mennyiségeken megjelenítésen kívül megelőző karbantartási javaslatot is tegyen a gép állapotromlásáról/kopásáról az idő elteltével. Ezeket az adatsorokat tanulóalgoritmus értékeli ki, metrikát készít belőle, ami az idő múlásával tendenciák figyelhetőek meg az eredeti betanult optimum értékekhez képest.

Az eredeti projektet Kereszty Márk készítette 2023-ban, ebből több terv, modell, adatgyűjtési és feldolgozási rész lett felhasználva és tovább fejlesztve, de ezekre a megfelelő részekenél kitérek.[9]

2.1. Architektúrális felépítés

A szenzorok és feldolgozó mikrovezérlők a Budapesti Műszaki és Gazdaságtudományi Egyetem **G épületében** található, Hagyományos forgácsolási labor két **E3N-01** típusú eszterga gépeire vannak felszerelve, innen küldik a mérésadatokat. Az eredeti projekt tanári esztergagépre lett felszerelve, ehhez társult még egy egység egy hallgatói gépre. A beérkező adatok feldolgozásáért, eltárolásáért és megjelenítéséért szolgáló Docker konténeres alkalmazás az I épület földszintjén található Ipar 4.0 Technológiai Központ 1-es szerverén található, az MQTT bróker, az adatbázis, backend feldolgozási modul, szimulációs API vezérlő és a műszerfal. A rendszer felépítését az [2.1. ábra](#) szemlélteti.



2.1 ábra - IoT rendszer felépítése

3. Mérőegység felépítése

3.1. Hardverelemek szerepei

A rendszer központi egységét a két mikrovezérlő alkotja, ezek bonyolítják le az adatok előkészítését és biztosítják azok áramlását. Ezen kívül a rendszernek része egy MEMS gyorsulásmérő, amellyel a főorsó rezgéseit lehet figyelni, egy optocsatoló amivel a főorsó fordulatszámát lehet meghatározni, egy infravörös hőmérsékletmérő, amivel a villamosmotor hőmérséklete mérhető és egy háromfázisú árammérő, ami a villamosmotor áramfelvétele mutatható ki.

3.1.1. Mikrovezérlők

A feladat megvalósításához két darab ESP32 alapú mikrovezérlő szolgál, ezek az eredeti projektben már bizonyítottak, így nem volt szükség másra váltani. A konkrét típus a NodeMCU ESP-WROOM-32S.

3.1.2. Szenzorok és egyedi mérőkészülékek

A rendszer az elődjéhez hűen, 4 szenzortól gyűjt adatot, amik az esztergagép 4 legrelevánsabb tényezőit foglalják magukba. Szenzorválasztásnál továbbra is volt egy feltétel, miszerint a munkaeszközt eredeti állapotában kellett hagyni. Mérendő mennyiségek és a választott szenzorok a [3.1. táblázatban](#) találhatók. A szenzorok közül a gyorsulásmérő, illetve a hőmérő I²C-n keresztül csatlakozik a mikrovezérlőkhöz, az optocsatolónak a tápellátáson kívül egy digitális, az árammérőnek pedig négy analóg bemenetre van szüksége. A modularitást az adatgyűjtő alján található horizontális XH+JST csatlakozók teszik lehetővé, ezzel elkerülhető a közvetlen forrasztás.

KOMPONENS	SZENZOR
FŐORSÓ REZGÉSEI	MPU9250 MEMS gyorsulásmérő, giroszkóp és magnetométer
FŐORSÓ FORDULATSZÁMA	IR infravörös réses optikai sebességmérő érzékelő (optocsatoló)
FŐORSÓMOTOR HŐMÉRSÉKLETE	MLX90614 érintésmentes hőmérő
FŐORSÓMOTOR ÁRAMFELVÉTELE	ZMCT103C áramváltó transzformátor (3x)

3.1.2 táblázat - Mérendő mennyiségek és a hozzájuk választott szenzorok

Az eredeti gyorsulásmérő MPU 6250 helyett, MPU 9250-M került beszerzésre, mivel hazai piacon már kevésbé forgalmazzák. A 9250-es modell közvetlen fejlesztése a 6250-es egységnek, így elődje összes tulajdonságát hordozza, plusz extra magnetométerrel van felszerelve. A szenzor rögzítési módja a [3.1.2/1 ábrán](#) látható; az érzékelő a főorsóház közelében került elhelyezésre.



3.1.2/1 ábra – Felül az MPU9250-M szenzor alul pedig az IR réses optocsatoló látható az eszterga főorsójának végén a burkolat alatt

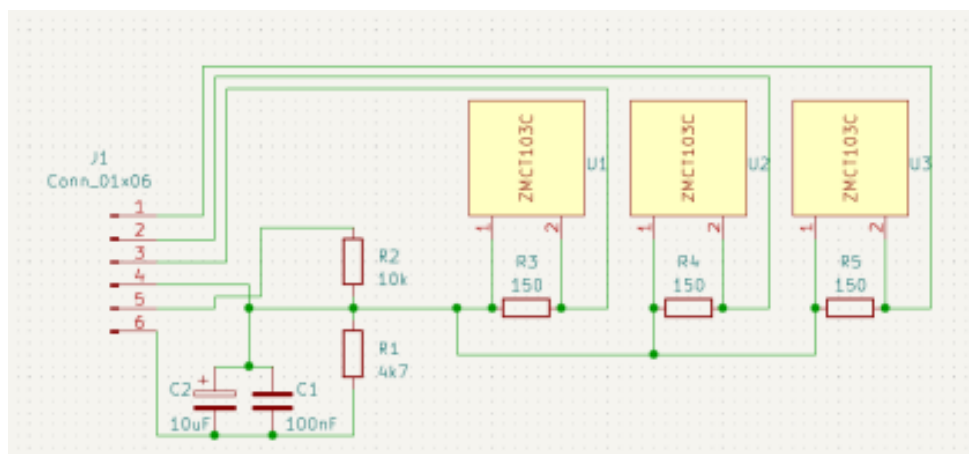
A fordulatszámot az IR infravörös réses optocsatoló és a hozzá készített főorsóra rögzített tárcsa segítségével mérhető meg. A táp és föld kimenet mellett van egy digitális kimenet, amihez „interrupt” köthető be amit a tárcsán lévő réseken áteső fény vált ki, vagyis a szenzor nyitott állapota. Ez fentebb lévő [3.1.2/1 ábrán](#) a főorsó végéhez közelebb eső szenzor.

A motor hőmérsékletének meghatározására az MLX90614 típusú, érintésmentes infravörös hőmérő került ismét alkalmazásra. A mérések során előfordulhatnak eltérések a valós hőmérsékleti értékekhez képest, amelyeket elsősorban a felületen megjelenő zsíros szennyeződések okozta infravörös reflexió eredményezhet. Ennek minimalizálása érdekében a mérési felület rendszeres tisztítása szükséges, ami a gépkezelő felelőssége. A főorsómotor irányába elhelyezett hőmérő a [3.1.2/2 ábrán](#) látható.



3.1.2/2 ábra – A villamosmotor fölé helyezett MLX90614 típusú hőmérő szenzor a burkolat alatt

Az eredeti projektben lévő ZMCT103C transzformátor, illetve áramkörelemek és egyedileg nyomtatott nyáklapot használtam fel újabb árammérő egységek építéséhez. Az összeépített mérőegység az eredeti 3D nyomtatott házában kapott helyet. Az árammérő kapcsolási rajza a [3.1.2/4 ábrán](#) látható, ami az eredeti projekt 30. oldalán is szerepel. [9]



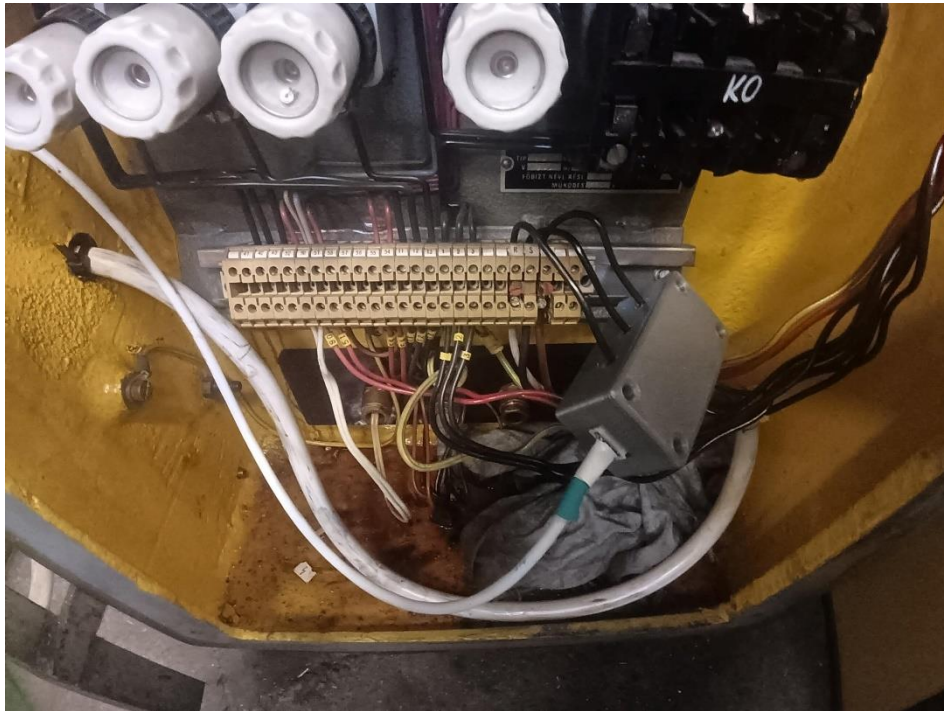
3.1.2/4 ábra – Háromfázisú árammérő egység sematikus rajza

Az árammérő egység működéséről az eredeti készítő szakdolgozatának 30-31-edik oldalán lehet olvasni.[10] Egyedileg készített nyák és lapraszerelt THT modulok az eredeti terv alapján készült el, ami [3.1.2/3 ábrán](#) látható.



3.1.2/3 ábra – Az egyedileg készített nyáklap és ráforrasztott elektronikai komponensek

A vezetéket egy XH csatlakozón lehet az árammérőhöz kötni, továbbá tartozik hozzá egy 3D nyomtatott ház is, ami szintén az eredeti tervek alapján került kialakításra. A beszerelt egység a [3.1.2/4 ábrán](#) látható.



3.1.2/4 ábra - A villamosmotor 3 fázisán (RST) átvezetett árammérő modul

Továbbá a nyomtatott áramköri lapon két nyomógomb, valamint egy SSD1306 vezérlővel ellátott, 128×64 képpont felbontású OLED kijelző került elhelyezésre.

A nyomógombok segítségével a belső menürendszerben történő navigáció történik, míg a kijelzőn megjeleníthető a rendszer aktuális állapota, valamint az esetleges hibák forrására vonatkozó információk. Így a gépkezelő közvetlen visszajelzést kap a rendszer működéséről.

3.1.2.1. Előzetes mérési pontok

Előzetes szenzorelhelyezés alapján építettem ki az újabb rendszert, így az alábbi szempontokat vettem figyelembe a telepítéshez:

- Az érzékelő a főorsó csapágáihoz minél közelebb kerüljön, így a rezgések a keletkezési helyükhöz közeli ponton legyenek mérhetők. A főorsó hajtás felőli végénél, a burkolat mögött található a gyorsulásmérő, valamint az optocsatoló.
- Az árammérő a gyorsulásmérő alatt elhelyezkedő villamos elosztószekrényben került beépítésre.
- A hőmérő egy kábelvezető elem támogatásával lett a motor fölött elhelyezve.

3.1.2.2. Szenzorok rögzítése, egyéb alkatrészek

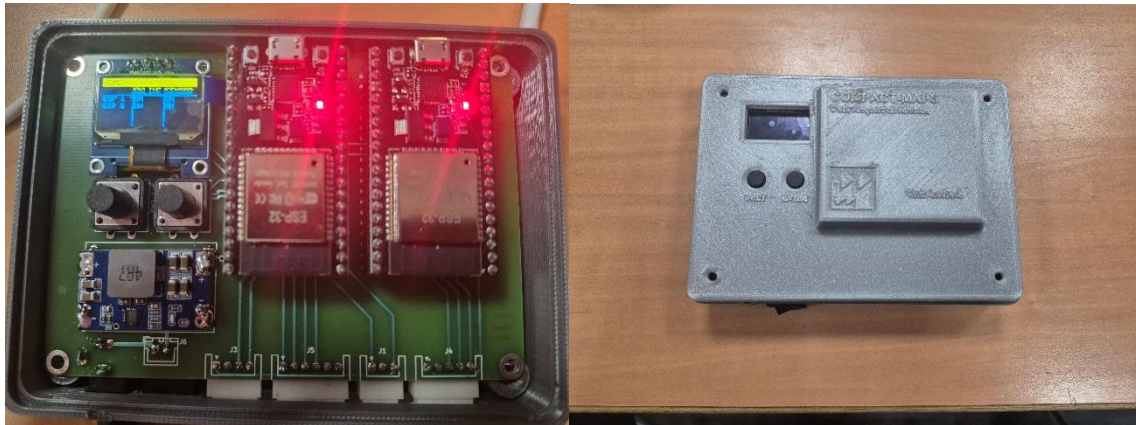
A rendszer kialakításához összesen két adapter alkalmazása volt szükséges: az egyik a rezgésmérő és az optocsatoló rögzítését biztosítja, míg a másik a hőmérő pozicionálására szolgál közvetlenül a motor felett. Az adapterek 3D nyomtatással készültek, PETG alapanyag felhasználásával, mivel indokolt a fizikai tartósság. A nyomtatványok Anycubic Kobra 3 v2 nyomtatóval készültek, 80°C ágyhőmérséklettel és 240°C nyomtatófejhőmérsékleten. További elemek, mint az impulzusokat szolgáltató tárcsa és árammérő dobozolása is hasonló módon készült el. Egyéb részletek az eredeti készítő szakdolgozatának 32. oldalán találhatóak meg. [10]

3.1.3. Tápellátás

Tápellátás működéséről bővebben az eredeti készítő szakdolgozatának 32. oldalán lehet olvasni. [10]

3.1.4. Nyomtatott áramkör és ház

A mikrovezérlők, perifériák, tápellátó egységek és csatlakozók egy közös nyomtatott áramköri NYÁK lapon kerültek elhelyezésre, erről bővebben az eredeti szakdolgozat 33. oldalán lehet olvasni. [10] A NYÁK és ráforrasztott elemek az eredeti 3D nyomtatott házba kerültek, feliratátalakítás után. A 3.1.4 ábrán a nyomtatott áramkör és az átalakított szövegezésű ház.



3.1.4 ábra - Bal oldalon az összeszerelt nyáklap és THT komponensei, jobb oldalon a fedlappal ellátott verzió

3.2. ESP32 firmwarek

A rendszer teoretikus működése és ESP szerepkörök az eredeti szakdolgozat 34. oldalán lehet többet megtudni. [10]

3.2.1. ESP1 taszkjai

Az ESP1 működése több, egymástól elkülönített taszkra bontva valósult meg, kihasználva a mikrovezérlő kétmagos architektúrájából adódó párhuzamos feldolgozási lehetőségeket. A rendszerben több dedikált taszk került kialakításra, amelyek meghatározott funkciókért felelősek, így kisebb egységekre sikerült bontani a működést.

A **DisplayTask** feladata a kijelző tartalmának kezelése. Ez egy üzenetsoron (queue) keresztül kapja a megjelenítendő információkat, így a kijelző frissítése nem blokkolja a többi folyamatot, ezzel a grafikus felület frissítése elkülönül a logikai működéstől.

A **ButtonHandler** taszk a felhasználói bemenetek feldolgozását végzi. A gombnyomások megszakítások (interruptok) segítségével kerülnek érzékelésre, majd egy „kritikus szakaszon” keresztül kerülnek feldolgozásra [1], biztosítva az adatintegritást több taszk egyidejű futása mellett. Ez a taszk felel továbbá a menürendszer állapotkezeléséért és a navigációs események feldolgozásáért, valamint az ESP1 állapotának folyamatos ellenőrzéséért is.

A **ReadandSendSensorData** taszk külön magon fut magasabb prioritással, mivel időkritikus feladatokat lát el. Ide tartozik a szenzoradatok (áramfelvétel, fordulatszám és hőmérséklet) gyűjtése, valamint azok továbbítása MQTT protokollon keresztül. A taszk emellett kezeli az ESP2-vel való soros kommunikációt is beleértve az állapotinformációk fogadását, feldolgozását és az esetleges hibák detektálását.

A **StatusTask** feladata a rendszer állapotának periodikus jelentése. Ez magában foglalja a kapcsolat ellenőrzését, valamint *heartbeat* jelek küldését, amely alapján a szerver felől nyomon követhető a vezérlő(k) állapota.

3.2.2. ESP2 taszkjai

Az ESP2 működése szintén több, egymástól elkülönített taszkra bontva valósult meg, amely lehetővé teszi a rezgésadatok nagy frekvenciájú mintavételezésének és a hálózati kommunikációnak a párhuzamos kezelését.

A **CollectVibration** taszk a legmagasabb prioritással rendelkező taszk, amely a gyorsulásmérő által generált Data Ready megszakításokra reagál. A mintavételezés során az MPU9250 szenzorból származó gyorsulási adatok kerülnek kiolvasásra I²C kommunikáción keresztül, majd tömbökben kerülnek eltárolásra. Aztán kisebb 128-as chunk-okba rendeződnek az eredeti 1024-es méretű minták, amik elé metrikaként bekerül a kezdőindex és chunkméret is, amit a szerveroldalon újraépítek.

A **SendMqttMessage** taszk feladata a begyűjtött adatok továbbítása MQTT protokollon keresztül. Amennyiben elegendő mennyiségű minta gyűlt össze, az adatok JSON formátumban, darabolva kerülnek elküldésre, figyelembe véve az üzenetméret korlátokat. A kommunikáció során *mutex* védi az MQTT kliens elérését, így megelőzhető az egyidejű hozzáférésből adódó hibák. Ez a taszk szintén kezeli a kapcsolat fenntartását a brókerrel.

A **SerialMessageHandler** taszk az ESP1 és ESP2 közötti soros kommunikáció feldolgozásáért felel. Feladata a beérkező parancsok értelmezése és végrehajtása például az adatgyűjtés ki és bekapcsolása vagy a rendszer újraindítása. A kommunikáció egy egyszerű, ellenőrzőösszeggel védett protokollon keresztül történik, erről bővebben később beszélek.

A **SendSensorStatus** taszk a rendszer állapotának felügyeletét és jelentését végzi. Ennek során ellenőrzésre kerül az MQTT kapcsolat állapota, szenzor működőképessége, valamint az ESP1 elérhetősége. Az eredmények egyrészt MQTT-n keresztül kerülnek publikálásra, másrészt egy strukturált státuszüzenet formájában az ESP1 felé is továbbításra kerülnek. Ez lehetővé teszi a rendszer globális állapotának nyomon követését és a hibák gyors detektálását, valamint osztott felelősséggé.

3.3. Adatgyűjtés

Az adatgyűjtés az inicializáló rész, majd az MQTT kapcsolat felépítése után 1,5-2 másodpercenként történő esemény, mind a két ESP-n függetlenül egymástól következik be. Megfelelő topic-ra küldés után a folyamat újraindul.

3.3.1. Motorhőmérséklet mérése

A mérési folyamat során a szenzor által szolgáltatott objektumhőmérséklet kerül kiolvasásra Celsius fokban. Ehhez elődöm megoldása alapján az **Adafruit_MLX90614** könyvtárat használok fel. Az adatok érvényességének ellenőrzése több lépésben történik: egyrészt vizsgálatra kerül, hogy a szenzor inicializálása sikeres volt-e, másrészt a mért értékek tartományellenőrzésen esnek át. Amennyiben a kapott hőmérsékleti érték kívül esik a reális tartományon (például $-40\text{ }^{\circ}\text{C}$ alatti vagy $300\text{ }^{\circ}\text{C}$ feletti érték), illetve nem szám (NaN) esetén a rendszer hibás mérésnek kezeli.

3.3.2. Háromfázisú motor áramfelvételének mérése

A mérések hardveres időzítő által generált megszakítás segítségével történik, amely 5 ms-enként aktiválódik, így a mintavételezési frekvencia 200 Hz. Ennek eredményeként a rendszer másodpercenként 200 mintát vesz minden fázis áramjeléből. A folyamat során először az ESP32 analóg-digitális átalakítója (ADC) beolvassa az egyes fázisokhoz tartozó feszültségjeleket, valamint referenciajelként szolgáló *offset* értéket. Az *offset* levonásával a rendszer kiszűri a mérőáramkör egyenfeszültségű eltolását, így csak a tényleges váltakozó áramú komponens kerül feldolgozásra.

A kapott feszültségértékek az áramváltó ellenállásának figyelembevételével áramerősséggé lesz átszámítva. Az egyes mintákból a rendszer nem közvetlen átlagot számít, hanem az **effektív** (RMS - Root Mean Square) **áramerősséget** határozza meg. Az RMS érték meghatározásakor a négyzetösszeg elosztásra kerül a minták számával és a kapott átlag négyzetgyökével.

Kísérleti funkcióként bekerült, egy hibadetekciós funkció is az árammérő kapcsán, ami a jelenlegi működése alapján, szenzorhibát jelez, ha abszolút zéró terhelés esik rajta és üresjáratot, ha 0.05 alatt van a fázisokra jutó áramfelvétel.

3.3.3. Főorsó fordulatszám mérése

A fordulatszámmerési folyamat nem tér el jelentősen az eredeti projekthez képest, így erről az eredeti projekt 37-38. oldalán lehet információt találni. [10]

Kialakításra került egy alapvető hibadetektálási mechanizmus a szenzor ellenőrzésére. Amennyiben egy meghatározott időn belül nem érkezik új impulzus, a rendszer képes megkülönböztetni az álló állapotot és az esetleges szenzorhibát, figyelembe véve az áramfelvételi adatokat. Tehát, ha mérhető jelentős áramfelvétel de nem érzékelhető megszakítás az szenzorhibát jelez, üresjáratú áramfelvétel esetén pedig nem történik forgás.

3.3.4. Főorsó rezgésének mérése

Rezgésmérés során az ESP32 a MPU9250 mérőegység gyorsulásmérőjének adatait olvasom ki. A szenzor az X, Y és Z tengelyek mentén fellépő gyorsulásértékeket szolgáltatja **m/s²** egységben, amelyek a gép működése közben keletkező rezgések jellemzésére felhasználható. Mérésről bővebben az eredeti projektbe bővebben a 38-39. oldalon lehet olvasni. [10]

Újdonság azonban, hogy a tényleges adatkiolvasás a **CollectVibration** taszkban történik, amely az **i2cMutex** használatával biztosítja a busz kizárólagos elérését.

Az adatküldés a **SendMqttMessage** taszkban történik, amely ellenőrzi a gyűjtés állapotát és a hálózati kapcsolat elérhetőségét. A nagy adatmennyiség miatt a mért értékek nem egyetlen üzenetben kerülnek továbbításra, hanem kisebb, előre definiált méretű blokkokra bontva. A **publishChunk** függvény JSON formátumú üzeneteket állít elő, amelyek tartalmazzák az aktuális adatblokk kezdőindexét, méretét, valamint magukat az adatokat egy tömbben 2 byte-ú int típusként (eredeti 4 byte-os float).

3.4. Karbantartási felület

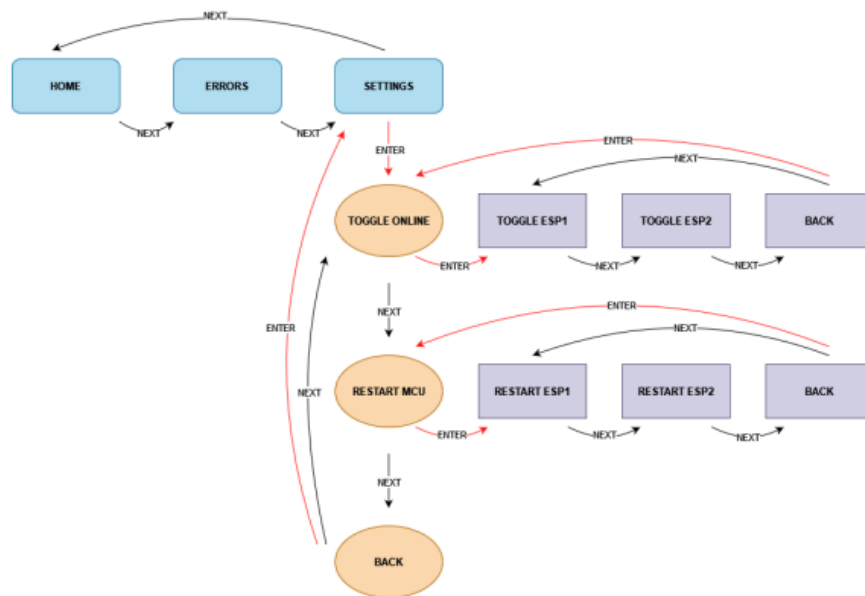
3.4.1. A menürendszer működése

Eredeti működése szerint rekurzív hívások sora volt az adott menüsztben való léptetés, amely áttekinthetetlen kódokhoz és instabil villódzó és széteső képernyőhöz vezetett. További architektúrális hibát jelentett, hogy a kijelző és hőmérsékletmérő egyazon I²C vezetéket hasznosította arbitráció nélkül, ami mindkét elem működését kompromittálta. Helyes működés jegyében I²C *mutex* általi arbitráció került bevezetésre. A menürendszer az alábbi módon lett átalakítva:

Az új menürendszer egy véges állapotgép (FSM) elvén alapul, amelyben az egyes képernyők külön állapotokat reprezentálnak, a felhasználói interakciók pedig állapotátváltásokat idéznek elő. A rendszer három fő nézetet tartalmaz: a kezdőképernyőt (**HOME**), a hibajelentő nézetet (**ERRORS**), valamint a beállítási menüt (**SETTINGS**). Az állapotok közötti váltás a **NEXT** gombbal történik, míg az **ENTER** gomb az adott állapoton belüli műveletek végrehajtására szolgál.

A **HOME** képernyő egy áttekintő felület, amely táblázatos formában jeleníti meg az ESP1 és ESP2 állapotát, külön jelezve a kommunikációs és szenzorműködést. Az állapotok dinamikusan frissülnek a beérkező adatok és hibajelzések alapján. Az **ERRORS** nézet célja a rendszerben fellépő hibák listázása, ahol az aktív hibák kerülnek megjelenítésre. A **SETTINGS** menü több szintből álló struktúrát valósít meg. A főmenüben két fő funkció érhető el: az adatgyűjtés ki és bekapcsolása, valamint az eszközök újraindítása. Ezekhez külön almenük tartoznak, ahol az ESP1 és ESP2 külön-külön vezérelhető. A műveletek végrehajtása után a rendszer visszajelzést ad ideiglenes „Working...” üzenet megjelenítésével.

A menürendszer működése nem blokkoló módon került kialakításra. Az időigényes műveletek végrehajtása közben a felhasználói felület továbbra is reszponzív marad, amit időzített állapotkezelés biztosít. Az állapotgép működését az alábbi 3.4.1 ábra szemlélteti, amely vizuálisan is bemutatja az egyes képernyők közötti átmeneteket és az elérhető műveleteket. Az ábra az eredeti szakdolgozatban is megtalálható a 41. oldalon. [10]



3.4.1 Menürendszer állapotgép modellje

3.4.2. Kirajzolás

A kijelző tartalmának megjelenítése strukturált módon, képernyőnként elkülönített függvények segítségével történik. Minden nézethez külön kirajzoló függvény tartozik, amely felelős az adott képernyő teljes tartalmának előállításáért. A megjelenítés egy üzenetsor alapú mechanizmuson keresztül történik, amely biztosítja, hogy a kijelző frissítése ne akadályozza a rendszer egyéb működését.

A megjelenítés első lépéseként a kijelző törlése történik, majd egy fejléc kerül kirajzolásra, amely az aktuális menüpont nevét tartalmazza. A fejléc kiemelése inverz színezéssel történik, ami vizuálisan elkülöníti azt a tartalomtól. Ezt követően kerül sor az adott képernyő specifikus elemeinek megjelenítésére.

A **HOME** nézet esetében egy rácsszerkezet került kialakításra vonalak segítségével, amelyben a különböző státuszok jelennek meg. Az adatok dinamikusan frissülnek, így a kijelző mindig az aktuális állapotot tükrözi. Az oldal tartalma a 3.4.2/1-es ábrán látható.



3.4.2/1 ábra – Home oldalon látható hibamentes működés

Az **ERRORS** képernyő listanézetet alkalmaz, ahol csak az aktív hibák jelennek meg szükség esetén két oszlopba rendezve, hibamentes állapotban pedig „NO ERRORS” üzenet olvasható. Az ERRORS képernyő állapotai a 3.4.2/2-es ábrákon láthatóak.



3.4.2/2 ábrák – Bal képen nincs érzékelt hiba, míg a jobb oldalon kommunikációs és rezgésadatgyűjtési hiba lépett fel

A **SETTINGS** menüben a navigációt egy kurzor jelzi, amely a kiválasztott sor előtt jelenik meg. A kurzor pozíciója a felhasználói bemenetek alapján változik. Az almenükben az aktuális állapotok (ON/OFF) szöveges formában kerülnek megjelenítésre. A SETTING menü funkcióválasztó képernyője a 3.4.2/3-mas ábrán látható.



3.4.2/3 ábra – A műveletválasztó főképernyője

3.5. Kommunikáció a rendszerben

3.5.1. ESP-k közötti kommunikáció

Az ESP-k közötti fizikailag kialakított kommunikációról az eredeti szakdolgozat 44. oldalán lehet tájékozódni. [10]

A logikai szintű kommunikáció átdolgozása is indokolt volt, mivel fejlesztés közben nagyon instabil volt a működése, ezért hibaellenőrző és hibatűró megoldás kialakítására volt szükség.

3.5.1.1. ESP1 → ESP2 parancsok

Az ESP1 egyszerű, hárombyte-os üzeneteket küld, amik összetétele az alábbi:

[HEADER | CMD | CRC], ahol:

- HEADER = 0xAA
- CMD = parancskód
- CRC = egyszerű XOR ellenőrző ($CMD \oplus 0xAA$)

KÓD	JELENTÉS
0X00	Heartbeat (kapcsolat fenntartása)
0X01	Adatgyűjtés ki-/bekapcsolása
0X02	ESP2 újraindítása

3.5.1.1 táblázat - ESP1 által küldött üzenetek és azok jelentése

A parancsok elküldése a **sendCommand()** függvényen keresztül történik, amely gondoskodik a megfelelő keretezésről és az ellenőrző összeg kiszámításáról. Az ESP2 oldalon a **checkSerialMessage()** függvény egy állapotgépet valósít meg, amely byte-onként dolgozza fel a beérkező adatokat. A feldolgozás lépései:

1. HEADER ellenőrzése (0xAA)
2. Parancskód beolvasása
3. CRC validálása
4. Parancs végrehajtása (handleCommand())

A **handleCommand()** függvény végzi a tényleges műveleteket:

- 0x01 esetén: az enableCollection logikai változó invertálása (adatgyűjtés engedélyezése/tiltása)
- 0x02 esetén: az ESP2 újraindítása (ESP.restart())

3.5.1.2. ESP2 → ESP1 állapotüzenetek

Az ESP2 összetettebb struktúrában küld visszajelzést. Az üzenet egy struktúra formájában kerül továbbításra:

```
struct Esp2Status {  
    uint8_t header;  
    uint8_t version;  
    uint8_t flags;  
    uint32_t uptime;  
    uint8_t checksum;  
};
```

Ahol a mezők az alábbi jelentéssel bírnak:

- **header (0xAA):** keretkezdő
- **version:** protokoll verzió
- **flags:** bitenként kódolt állapotinformáció
- **uptime:** működési idő milliszekundumban
- **checksum:** XOR alapú ellenőrző összeg

BIT	JELENTÉS
0X01	ESP2 online
0X02	MQTT kapcsolat aktív
0X04	MPU9250 működik
0X08	Hőmérséklet szenzor OK
0X10	Árammérés OK
0X20	Fordulatszám-mérés OK
0X40	Adatgyűjtés aktív

3.5.1.2 táblázat - ESP2 által küldött üzenetek és azok jelentése

A flag-ek lehetővé teszik, hogy egyetlen byte-on belül több állapotinformáció is továbbításra kerüljön. Az üzenetküldést a **sendStatus()** függvény végzi, amely:

1. Feltölti a struktúrát aktuális értékekkel
2. Kiszámolja a checksum-ot (calcChecksum())
3. Byte-szinten elküldi az adatot a soros vonalon

Az ESP1 a **readStatus()** függvény segítségével fogadja az adatokat. Ez egy puffert beolvasást valósít meg, amely:

- Figyeli a HEADER byte-ot,
- Összegyűjti a teljes struktúrát,
- Ellenőrzi a checksum helyességét.

Sikeres validáció után az adatokat az **updateEsp2()** függvény dolgozza fel, amely kitölti az esp2 állapotstruktúrát. Ez közvetlenül hat a felhasználói felületre (**HOME** képernyő státuszai). A rendszer tartalmaz egy időtúllépés kezelést is (**checkEsp2Timeout()**), amely offline állapotba helyezi az ESP2-t, ha egy meghatározott ideig nem érkezik friss adat felőle.

3.5.2. Mérésadatok küldése

Az alapvető MQTT küldés és paraméterezés változatlan maradt, ennek leírása a az eredeti szakdolgozat 46-47. oldalon található meg. [10]

Kiegészítésre került a topic-ok halmaza a státuszinformációk továbbításával, amely lehetővé teszi az eszközök működésének folyamatos felügyeletét. A funkció bevezetésével távolról is diagnosztizálhatók a felmerülő hibák a rendszerben és probléma esetén felkészülten lehet elvégezni a helyszíni szerelést. Előző rendszer esetén csak helyszíni paneeltávolítás után laborkörnyezetben lehetett megállapítani, hogy az ESP1 meghibásodott. Most, ha ez bekövetkezne, az ESP2 jelezné, hogy nem éri el az ESP1-et. Az ESP1 esetében a **publishEsp1Status()** függvény egy JSON struktúrát állít elő, amely tartalmazza:

- ESP1 működési állapotát (konstans True, ha tud küldeni),
- MQTT kapcsolat állapotát (True, ha tud küldeni),
- A hőmérséklet-, fordulatszám- és áramérzékelők státuszát (True vagy False a hőmérsékletnél, fordulatszámnál vagy RPM_OK vagy RPM_NO_ROTATION vagy RPM_SENSOR_FAULT, árammérőnél vagy CURRENT_OK vagy CURRENT_NO_LOAD vagy CURRENT_SENSOR_FAULT).

- Valamint az ESP2 elérhetőségére és működésére vonatkozó információkat (ESP2 Online True vagy False, ESP2 Collecting True vagy False, ESP2 MPU True vagy False).

Az ESP2 hasonló módon küld visszajelzést a **publishSystemStatus()** függvényen keresztül. Itt a JSON üzenet az alábbi kulcsinformációkat tartalmazza:

- ESP2 működési állapota (konstans True, ha tud küldeni),
- MQTT kapcsolat státusza (True, ha tud küldeni),
- Gyorsulásmérő (MPU) állapota (MPU True vagy False),
- Valamint az ESP1 elérhetősége (True, ha ESP1 felől jön utasítás).

3.6. Hiba és állapotdetekció

A rendszer megbízható működésének alapfeltétele a szenzorok és kommunikációs csatornák folyamatos felügyelete. Ezt mindkét mikrovezérlő külön-külön, de egymással összhangban végzi, így lehetővé válik a hibák gyors felismerése és kezelése. Nagyrészt az eredeti szakdolgozat metodikája alapján dolgoztam 1-2 kiegészítéssel. Eredetiről bővebben a 47. oldalon lehet megismerkedni. [10]

A fordulatszám érzékelésnél a **checkRpmSensor()** az utolsó megszakítás óta eltelt időt vizsgálja. Ha túl hosszú ideig nem érkezik jel, de közben az áramfelvétel alapján a motor működése feltételezhető, akkor szenzorhibát állapít meg. Ellenkező esetben a rendszert álló motorként értelmezi a rendszer. Az árammérés ellenőrzése a **checkCurrentSensor()** függvényben történik, amely az RMS értékek alapján dönt. Ha nincs elegendő minta vagy számítási hiba lép fel (NaN értékek), a szenzor hibásnak minősül. Nagyon alacsony áram esetén a rendszer „CURRENT_NO_LOAD” állapotot jelez, egyébként normál működést feltételez.

4. Szenzoradatok tárolás

A rendszer következő funkcionális egysége az ESP eszközök által gyűjtött adatok fogadása és tartós tárolása. Ennek megvalósítására az Ipar 4.0 Technológiai Központ 1. szerverén egy összetett környezet került kialakításra, amely különböző szolgáltatásokat integrál. A kommunikáció központi elemét egy MQTT bróker biztosítja, amely fogadja az ESP eszközök által küldött üzeneteket. Az adatok feldolgozását egy szerveroldali ágens és egy Python alapú szkript végzi, amelyek az üzeneteket lekérlik, majd az adatbázisba továbbítják.

Az egyes szolgáltatások elkülönített Docker konténerekben működnek, így a rendszer teljesen moduláris, egyes rétegek lecserélhetőek de biztosítva van az egyszerű üzemeltethetőség. A konténerek konfigurációját egy *docker-compose.yaml* állomány tartalmazza, amely egyes szolgáltatásokban található Dockerfile alapján építi fel a konténereket.

4.1. MQTT bróker

Az MQTT alapú kommunikáció központi eleme az MQTT bróker, amely a publikáló és feliratkozó kliensek közötti üzenetközvetítést végzi. Feladata a beérkező üzenetek fogadása, a megfelelő témakörök (**topicok**) kezelése, valamint az adatok továbbítása a feliratkozott eszközök és szolgáltatások számára. A rendszer megvalósítása során az **Eclipse Mosquitto** került alkalmazásra az eredeti kialakításhoz híven.

4.1.1. Bróker konfigurálása

Az MQTT bróker konfigurációja a *docker-compose.yaml* állományban definiált *mosquitto* szolgáltatáson keresztül valósult meg. A rendszerbiztonság növelése érdekében a bróker kizárólag hitelesített kliensek számára engedélyezi az üzenetküldést, ezért felhasználónév és jelszó alapú autentikáció került bevezetésre.

A bróker működési paraméterei a *mosquitto.conf* konfigurációs fájlban kerültek meghatározásra. Legfontosabb része a felhasználónév és jelszóalapú autentikáló sor, ami a *passwordfile.txt* tartalma alapján állítja be ezeket hash-elés után. A naplózási kimenet stdout irányba történő átállítása megkönnyítette a rendszer monitorozását és tesztelését.

A *listener* direktíva segítségével a rendszer az MQTT kommunikációhoz használt 1883-as porton figyel, míg az *allow_anonymous false* beállítás letiltja a hitelesítés nélküli kapcsolódásokat. A konfiguráció emellett tartalmazza a jelszófájl elérési útját is. Az elkészített konténer konfigurációjában meghatározásra került a szolgáltatás neve, az automatikus újraindítási szabálya, továbbá a szükséges környezeti változók és kötetcsatlakozások. A külső kliensek csatlakozásának lehetővé tétele érdekében a Docker porttovábbítás segítségével a konténer belső 1883-as portja a szerver egyik szabad portjához lett hozzárendelve.

4.1.2. MQTT Topic felépítés

A rendszerben minden mérési adat külön MQTT topichoz tartozik, azonban az eredetihez képes kisebb és egyszerűbb struktúra került kialakításra. A topicok az alábbi főbb elemekből épülnek fel:

- Telephely vagy rendszerazonosító | mérési környezet vagy üzem azonosítása
- Berendezés neve | adott gép vagy eszköz egyedi azonosítója
- Mérési kategória | például hőmérséklet, fordulatszám, áramfelvétel vagy rezgés
- Szenzor típusa | méréshez használt érzékelő megnevezése
- Mért paraméter | konkrét mérési adat vagy tengelyazonosító

A kialakított struktúra lehetővé teszi, hogy az egyes mérési típusok vagy berendezések egyszerűen szűrhetők legyenek akár wildcard alapú MQTT lekérdezésekkel is. Az alábbi példák a rendszerben alkalmazott topic felépítést szemléltetik:

MENNYISÉG	TOPIC STRUKTÚRA
MOTORHŐMÉRSÉKLET	factory/lathe01/temperature/mlx90614/tempC
FŐORSÓ FORDULATSZÁMA	factory/lathe01/speed/m0c70t3/rpm
MOTOR 3 FÁZISÁNAK ÁRAMFELVÉTELE	factory/lathe01/current/zmct103c/amp
FŐORSÓ X TENGELYŰ REZGÉSE	factory/lathe01/vibration/mpu9250/vibX
FŐORSÓ Y TENGELYŰ REZGÉSE	factory/lathe01/vibration/mpu9250/vibY
FŐORSÓ Z TENGELYŰ REZGÉSE	factory/lathe01/vibration/mpu9250/vibZ

4.2. Adattárolás

Elődjéhez híven az új rendszer is InfluxDB-t használ a szenzoradatok tárolásához. Ebből ismét az InfluxDB 2.x verzió került alkalmazásra, amely beépített webes kezelőfelületet, API-támogatást és fejlett hitelesítési lehetőségeket biztosít, míg az újabb 3.x verzió elsősorban nagyvállalati és felhőalapú környezetekre optimalizált architektúrát képvisel és már nem NO-SQL, hanem normál relációs adatbázis.

4.2.1. Influxdb dockeres szolgáltatásfelépítése

A *docker-compose.yml* állományban az MQTT bróker után az InfluxDB adatbázis szolgáltatás került definiálásra. A szolgáltatás és a konténer neve egyaránt *influxdb*, míg a használt konténerkép a 2.7.12-alpine verzió.

A szolgáltatás konfigurációja során az MQTT brókerhez hasonlóan engedélyezésre került az automatikus újraindítás (*restart: always*), ezzel a szerver újraindítása után minden szolgáltatás elindul. Az adatbázis tartós adattárolását Docker-kötet biztosítja, amely az adatállományok konténeren kívüli megőrzéséért felel. Az InfluxDB webes felületének és API szolgáltatásainak eléréséhez a konténer belső 8086-os portja került hozzárendelésre a szerver egyik szabad portjához.

A rendszer inicializálásához több környezeti változó is meghatározásra került, amelyek lehetővé teszik az adatbázis automatikus konfigurálását a konténer első indításakor. Ezek közé tartozik az inicializálási mód, a felhasználónév és jelszó, a szervezet neve, az alapértelmezett bucket, az adatmegőrzési idő, valamint az adminisztrátori token konfigurációja.

4.2.1.1. A bucket paraméterei

Az InfluxDB inicializálása során a rendszer automatikusan létrehozza az alapértelmezett bucketet a környezeti változókban meghatározott névvel és konfigurációval. A bucket létrehozásakor megadható az adatok megőrzési ideje (*retention policy*), amely meghatározza, hogy az adatbázis mennyi ideig tárolja a bejegyzéseket automatikus törlés előtt. Az optimális *retention* idő kiválasztása a felhasználási területtől és a rendelkezésre álló tárhelykapacitástól függ. A kialakított konfigurációban az adatbázis automatikusan törli a 30 napnál régebbi adatokat. Az adatbázis sikeres inicializálását követően további bucketek létrehozása is lehetséges, akár a grafikus kezelőfelületen, akár parancssori eszközök használatával is.

4.3. Szerveroldali ágens - Telegraf

A Telegraf az InfluxData által fejlesztett, nyílt forráskódú szerveroldali ágens, amely különböző adatforrásokból származó információk gyűjtésére, feldolgozására és továbbítására alkalmazható. Az IoT alapú adatgyűjtő rendszerekben kiemelt szerepet tölt be, mivel támogatja az ipari kommunikációs protokollok és szolgáltatások széles körét. A kialakított rendszerben a Telegraf feladata az MQTT brókeren keresztül érkező szenzoradatok fogadása, majd azok strukturált formában történő továbbítása az InfluxDB adatbázis felé. Az ágens moduláris felépítésének köszönhetően egyszerűen konfigurálható, továbbá lehetőséget biztosít egyedi bővítmények és adatfeldolgozási műveletek integrálására is.

4.3.1. Pluginok

A Telegraf működése pluginalapú architektúrára épül, amely lehetővé teszi az adatáramlás rugalmas konfigurálását. A rendszer négy fő plugintípust támogat: Input, Output, Processor és Aggregator. Az Input és Output pluginok biztosítják az adatok fogadását és továbbítását, így ezek használata elengedhetetlen az ágens működéséhez.

4.3.1.1. Input pluginok

A Telegraf input pluginjai biztosítják a különböző forrásokból érkező adatok fogadását és előfeldolgozását. Az ipari IoT rendszerekben ezek a komponensek felelősek a szenzoradatok begyűjtéséért, valamint azok strukturált továbbításáért az adatfeldolgozó és adattároló rétegek felé. A kialakított rendszerben az adatforrás szerepét az MQTT kommunikációs infrastruktúra tölti be, amelyen keresztül a különböző érzékelők és mikrokontrollerek valós időben publikálják a mérési adatokat. A Telegraf az *mqtt_consumer* input plugin alkalmazásával csatlakozik az MQTT brókerhez, majd a meghatározott topic struktúrák alapján fogadja és feldolgozza az üzeneteket. A használt paramétereket a [4.3.1.1 táblázat](#) foglalja magába funkciókkal párosítva.

PARAMÉTER	SZEREPE
SERVERS	Az MQTT bróker elérési címének meghatározása.
TOPICS	Azoknak a topicoknak a megadása, amelyekről az ágens adatokat fogad.
USERNAME	Felhasználónév az MQTT hitelesítéshez.
PASSWORD	Jelszó az MQTT kapcsolat biztonságos létrehozásához.
QOS	Az MQTT üzenetküldés megbízhatósági szintjének beállítása.
CONNECTION_TIMEOUT	A kapcsolatfelépítés maximális várakozási ideje.
DATA_FORMAT	A fogadott adatok formátumának meghatározása.
DATA_TYPE	Az adatok típusának definiálása a feldolgozás során.
NAME_OVERRIDE	Az adatbázisban tárolt mérési név felülírása.
TOPIC_PARSING	A topic szerkezetének elemzése és címkék létrehozása.
JSON_V2	JSON formátumú üzenetek mezőinek feldolgozása.

4.3.1.1 táblázat - telegraf.conf legfontosabb paraméterei

A konfigurációban több egymástól elkülönített input plugin került definiálásra a különböző típusú adatok kezelésére. A hőmérséklet és fordulatszámadatok lebegőpontos értékként kerülnek feldolgozásra, ezért ezek esetében a *data_type = "float"* beállítás került alkalmazásra. Az áramfelvételi és rezgés adatok karakterláncként érkeznek, így azok feldolgozása *string* adattípussal történik. Az egyes inputok külön mérési névvel rendelkeznek, amelyet a *name_override* paraméter biztosít az adatok egyszerűbb azonosítása érdekében.

A rendszer állapotinformációinak feldolgozására a *json_v2* parser került alkalmazásra. Ennek segítségével az MQTT üzenetekben érkező JSON struktúrák mezői különálló adatbázis mezökké alakíthatók. A konfigurációban meghatározott field elemek biztosítják az egyes változók típushelyes feldolgozását, például logikai vagy szöveges formában. A topicok feldolgozásában fontos szerepet tölt be a *topic_parsing* mechanizmus is. Ennek alkalmazásával az MQTT topic hierarchiájából automatikusan kinyerhetők az olyan metaadatok, mint a gép azonosítója, az érzékelő típusa vagy a mért paraméter neve. Az így létrehozott *tags* lehetővé teszik az adatok hatékony szűrését, csoportosítását és vizualizációját az adatbázis-kezelő és monitoring rendszerekben.

4.3.1.2. Output plugin

A Telegraf output pluginjai felelősek a feldolgozott adatok továbbításáért a kiválasztott adattároló vagy megjelenítő rendszer felé. A rendszerben az *outputs.influxdb_v2* plugin került alkalmazásra, amely az adatok továbbítását végzi az InfluxDB adatbázis felé. A kapcsolat konfigurálásához meg kell adni az adatbázis elérési címét az *urls* paraméter segítségével, valamint a hitelesítéshez szükséges *token* értéket. Az *organization* paraméter az adatbázis szervezeti egységét azonosítja, míg a *bucket* határozza meg az adatok tárolási helyét.

4.3.2. Ágens előzetes konfigurációja

A Telegraf működésének alapvető paramétereit az *[agent]* szekció tartalmazza, amely meghatározza az adatgyűjtés és adattovábbítás időzítését, valamint a belső puffereelési mechanizmus működését. Az *interval* paraméter szabályozza az adatgyűjtési ciklus időközét, amely jelen konfigurációban 1 másodperc. A *round_interval* beállítás biztosítja, hogy a mintavételezés pontos időhatárokhoz igazodjon. A *metric_batch_size* paraméter meghatározza az egyszerre továbbított adatmennyiséget, míg a *metric_buffer_limit* a helyileg tárolható metrikák maximális számát szabályozza kapcsolatmegszakadás esetén. A *flush_interval* paraméter az adatok adatbázisba történő továbbításának gyakoriságát definiálja, amely jelen esetben 5 másodpercenként történik.

4.3.2.1. Telegraf konténer beállításai

Harmadik szolgáltatásként szerepel a *telegraf*, konfiguráció során a *telegraf:1.29-alpine* image biztosítja a futtatókörnyezetet, míg a *restart: always* beállítás automatikus újraindítást tesz lehetővé hiba esetén. A *depends_on* paraméter garantálja az MQTT bróker és az adatbázis elérhetőségét induláskor. A *volumes* csatolás segítségével a külső konfigurációs fájl közvetlenül elérhető a konténer számára.

5. Adatelemzés és utófeldolgozás

Az adatelemzési és adatmegjelenítési folyamatok ismét egységes fejlesztési környezetben kerültek kialakításra, cél előtt tartva a jövőbeli továbbfejlesztési lehetőségeket a Data Science és ML algoritmusok téren, ahol vezető környezet a **Python**.

5.1. Backend modul felépítése

A backend modul a rendszer központi feldolgozóegységként működik, amely több, egymással párhuzamos feladatot lát el. Fő szerepe az MQTT kliens futtatása, amely fogadja a szenzoradatok által publikált nyers mérési adatokat, majd azokat továbbítja a megfelelő adatfeldolgozó függvények számára. A modul emellett gépi tanulási algoritmusok futtatását is végzi, amelyek lehetővé teszik az adatok elemzését, az anomáliák felismerését és a rendszerállapot kiértékelését. Előbbiről bővebben a saját fejezetében kerül majd sor.

A backend szolgáltatás konténerizált környezetben, *backend* néven került kialakításra. A Dockerfile tartalmazza a szükséges Python környezet létrehozását, a projektfájlok másolását, valamint a *requirements.txt* állományban definiált függőségek telepítését a *RUN* utasítás segítségével. A konfigurációban továbbá meghatározásra kerültek a környezeti változók, a szerver portbeállításai, az automatikus újraindítás módja, valamint a *depends_on* paraméter, amely biztosítja, hogy a backend szolgáltatás csak az MQTT bróker és az adatbázis elindulása után kezdje meg működését.

5.1.1. Adatfeldolgozás

A főprogram inicializálási kódrésze indulás után MQTT kliensként üzemel, beállítja a felhasználónév és jelszó kombinációt a brókerhez, feliratkozik az összes topic-ra egy *whitecard mask* alapján, majd megfelelő brókerhez és porthoz csatlakozik, végül a *client.loop_forever()* függvényhívással a végtelenségig futtatja magát. Közben, ha bármelyik topic-ra üzenet érkezik az *on_message* függvény meghívásával 2 fontos paramétert ad át a *process_message* főfüggvénynek. Első maga a *topic*, azaz hogy melyik szenzortól érkezett adat és a *payload*, ami a topic-ra érkezett üzenet.

A *process_message* ezután az üzenetet szétbontja és aszketusok alapján más feldolgozási módokat választ, ahol elkezdődik az utófeldolgozás.

5.1.2. Elemzési módszerek és algoritmusok

A rendszerben alkalmazott adatelemzési módszerek célja a forgácsolóberendezések állapotának folyamatos monitorozása, valamint az esetleges rendellenességek korai felismerése. Az elemzési folyamat alapját rezgés, áram, fordulatszám és hőmérsékletadatok együttes feldolgozása képezi. A különböző szenzorokból származó mérési adatokból statisztikai és frekvenciatartománybeli jellemzők kerülnek előállításra, amelyek alkalmasak a gép aktuális állapotának meghatározására és gépi tanulási modellek bemeneteként is felhasználhatók.

A rezgéselemzés kiemelt szerepet tölt be az ipari diagnosztikai rendszerekben, mivel a mechanikai hibák jelentős része a vibrációs jelek változásán keresztül már korai szakaszban felismerhető. A rendszer a háromtengelyű gyorsulásmérőből érkező mintákat dolgozza fel, amelyek csomagolt formában érkeznek MQTT üzeneteken keresztül. Az adatok összeállítását követően a rendszer több különböző jellemzőt számít ki.

Az egyik legfontosabb mutató az **RMS** (Root Mean Square) érték, amely a rezgésjel effektív amplitúdóját jellemzi. Az RMS érték növekedése általában a gép mechanikai terhelésének vagy kopásának emelkedésére utalhat. Az RMS számítás alapját az alábbi összefüggés adja:

$$RMS = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$$

Az RMS elemzés különösen alkalmas csapágyhibák, kiegyensúlyozatlanság és tengelyproblémák detektálására.

A rendszer frekvenciatartománybeli vizsgálatot is végez a gyors Fourier-transzformáció (**FFT**) alkalmazásával. Az FFT segítségével meghatározhatók azok a domináns frekvenciakomponensek, amelyek egy adott mechanikai hibára utalhatnak. A forgó alkatrészek meghibásodása gyakran meghatározott frekvenciákon okoz amplitúdónövekedést, ezért az FFT elemzés hatékony módszer a rezonanciajelenségek és periodikus rezgések azonosítására.

$$X(k) = \sum_{n=0}^{N-1} x(n)e^{-j2\pi kn/N}$$

Az algoritmus **Hamming-ablakozást** alkalmaz a spektrális szivárgás csökkentése érdekében, amely pontosabb frekvenciaelemzést tesz lehetővé.

A rendszer a teljes spektrum mellett meghatározza a legnagyobb amplitúdójú frekvenciakomponenst (`fft_peak`) is.

A vibrációs adatok további elemzése a teljesítményspektrum sűrűség (**PSD** - Power Spectral Density) számításával történik. A PSD segítségével meghatározható, hogy a rezgésenergia hogyan oszlik el az egyes frekvenciatartományok között. Ez különösen hasznos a zajos mérési környezetekben, mivel a spektrális energiaeoszlás alapján pontosabban elkülöníthetők az üzemszerű és hibás működésből származó komponensek.

Az áramelemzés során a rendszer háromfázisú árammérést végez, amelyből meghatározásra kerül az átlagos áramerősség (**current_mean**) és az áramkiegyensúlyozatlanság (**current_imbalance**). Az átlagos áramfelvétel jól jellemzi a gép aktuális terhelését, míg a fázisok közötti eltérések mechanikai vagy elektromos hibákra utalhatnak. Az áramkiegyensúlyozatlanság növekedése például csapágykopás, tengelyszorulás vagy motorhibák következménye lehet. Az elemzés előnye, hogy a villamos paraméterek változása gyakran hamarabb jelentkezik, mint a tényleges mechanikai meghibásodás.

5.1.3. Utófeldolgozás rögzítése

Miután megtörtént az szenzoradatok feldolgozása mindegyikhez egy külön `write`-tal kezdődő függvény tartozik, amik a sztenderd `InfluxDBClient` `write_api`-jával az adatbázisba írják rekord-jait. Az API az alábbi paramétereket várja el az adatbázisba íráshoz:

- InfluxDB URL
- InfluxDB Token
- InfluxDB Organization
- InfluxDB Bucket

6. Sztochasztikus szimulációból komplex vezérlés

A szerveroldali stack tesztelése az első pillanatoktól kezdve kulcsfontosságú eleme volt a projektnek. A szenzorbeszerzés is elhúzódó folyamat volt, így talán a 7. oktatási héttől tudtam azon a téren haladni. Így indokolt volt egy szimulációs modul fejlesztése, ami kezdetben véletlen adatokat generálva teszteli a rendszert, majd egyre inkább valósághűt közvetít. Ebben a szekcióban ennek a modulnak a fejlődését mutatom be.

6.1. Korai fázis

Mivel a projekt demonstrációs célokra is lett tervezve, viszont nem egész napom használják az éles rendszert, hanem inkább csak megmunkálás közben, így elődöm rendszeréhez híven demonstrációs célra szükség volt egy megoldásra, ami bemutatja a működést és ezzel együtt feltölti a felhasználói felület grafikonjait.

Célomhoz **FastAPI**-t használok, *threading* szálkezeléssel mivel egyszerre több egyidejű szimulációra skáláztam a működést. Működésük során, rengeteg iteráción mentek át, hogy konzisztensen tudjanak működni.

Az eszterga típusát megismerve a hivatalos gépkönyvből kivett adatok alapján véletlen számokat generált a modul egy tartományban. Ez a megoldás látványos volt a felületen, pláne a rezgésértékeken, de semmikép sem volt valósághű.

6.2. Fokozatok bevezetése

Miután megfigyeltem, hogy a véletlen értékek generálása nem megfelelő a tanulóalgoritmusok számára se, átálltam a 3 kategória alapú véletlenszám generálásra. Itt már elkülönítettem alacsony, közepes és magas fordulatszám alapon a modellt, azzal a gondolattal, hogy alacsony fordulatszámon az összes tényező alacsonyabb értéktartományban mozog, mint a magas esetén. Ez a megoldás jobbnak bizonyult, de tanulóalgoritmus számára még mindig kiszámíthatatlan volt, mivel a realisztikus szemlélet a rendszerek megfigyelhető kopása, ami lassú folyamat idővel növekvő kilengésekkel.

6.3. Kopásszimuláció

Miután fejlesztettem a tanulóalgoritmus működését, egy időben konzisztensebb modellre kellett átalakítani a szimulációs modult. Meghagytam a három kategóriába sorolást, de kiegészítettem egy kopásinjektáló résszel, ami az mennyiség értékeit jobban szórja az alaphoz képest, ezzel tesztelve a tanulóalgoritmus metrikai változásait. Ezzel a megoldással jobb lett a szimuláció, de továbbra is nagyon behatárolt működése volt, illetve közben megint fejlődött a tanulóalgoritmus, ami már valós adatokon alapult, így ismét indokolt volt a szimulátor átalakítása.

6.4. Végző megoldás

Egy újabb megközelítés vezérelt, ami a backend modulban már bizonyított, mégpedig, hogy 9 állapotra osztom fel a szimulátor bemenetét és egyes esethez egy hozzátartozó Excel fájlt rendelek. A kiolvasott adatok megfelelnek a valódi eszterga által rögzített értékekkel, így tehát a szimulátor újrajátssza őket. Állapotmódosító paramétereknek bekerült még a kopás ráta, ami meghatározza a kopás lineáris bekövetkeztét az idő múlásával, konkrét hiba injektálható mechanizmus névszerint:

- Imbalance
- Misalignment
- Bearing fault
- Looseness

Ezek a paraméterek különböző mértékben befolyásolják az adatfolyamot, így jól kimutatható mind a tanulóalgoritmusok által a változás, mint mindenféle grafikonon. Ehhez a paraméterhez is került romlaserősítő paraméter, ami segít tesztelni a rendszer az idő múlásával erősödő hibákkal.

Ez az iteráció felel meg, egy komplex tesztelő és demonstrációs modulnak, amivel már elemzéseket és következtetéseket is levonhatók lettek a tanulóalgoritmusok jósága és működése terén.

7. Felhasználói felület

7.1. Plotly Dash

Eredeti kialakítást szemelőt tartva, az alkalmazás adatvizualizációs modulja **Plotly Dash** keretrendszerrel épült újra. A Plotly Dash egy Python alapú webalkalmazás-fejlesztő keretrendszer, amelyet elsősorban interaktív adatvizualizációk és analitikai dashboardok készítésére használnak. A könyvtár a Python nyelvet ötvözi a modern webes technológiákkal, mint a Flask, React és Plotly.js. A Dash alkalmazások működésének alapját a *layout* struktúra adja, amely meghatározza az oldalon megjelenő elemek hierarchiáját és elrendezését. A felhasználói interakcióért a *callback* mechanizmus felel, ami függvények deklarálásával képes gombkattintásokra is reagálni. Ezek két részre oszlanak, **Input** és **Output** kategóriába. Az **Output** határozza meg, hogy melyik elem változik, míg az **Input** azt, hogy mely esemény váltja ki a frissítést. Grafikonok megjelenítésére a Dash a Plotly könyvtárat használja, amiben több mint 40 grafikon típust, beleértve a vonal-, oszlop-, hisztogram-, hőkép-, 3D és földrajzi térképeket.

Több eredeti funkció optimalizálva lett valamint, nagyban kiegészült a működése kezdetben a több esztergátámogatáson át az összetett szimulátor konfigurátor bevezetésén a tanulóalgoritmus metrikák publikálásán.

7.1.1. Adatelérés

Az eredeti projektben a backend modul feladata volt egy WebSocket szerver futtatása is, amire a Plotly Dash alkalmazás feliratkozik így kapta a feldolgozott szenzoradatokat, majd grafikonba illeszti. Az új megközelítésben a Plotly Dash alkalmazás meghatározott időközönként, 4 másodpercenként közvetlen lekérdezést indít az InfluxDB adatbázis felé. Mivel az alkalmazás nem igényel valódi (real-time) adatfrissítést milliszekundumos késleltetéssel, a *polling* alapú megoldás megfelelő teljesítményt biztosít, miközben egyszerűbb architektúrát és könnyebb karbantarthatóságot eredményez.

Ehhez a mostani backend modulhoz hasonlóan InfluxDBClient példány hozok létre és felparaméterezése után a megfelelő függvényekkel lekérem a feldolgozott szenzor adatokat, ML metrikákat és egyéb státuszadatot az ESP-k felől.

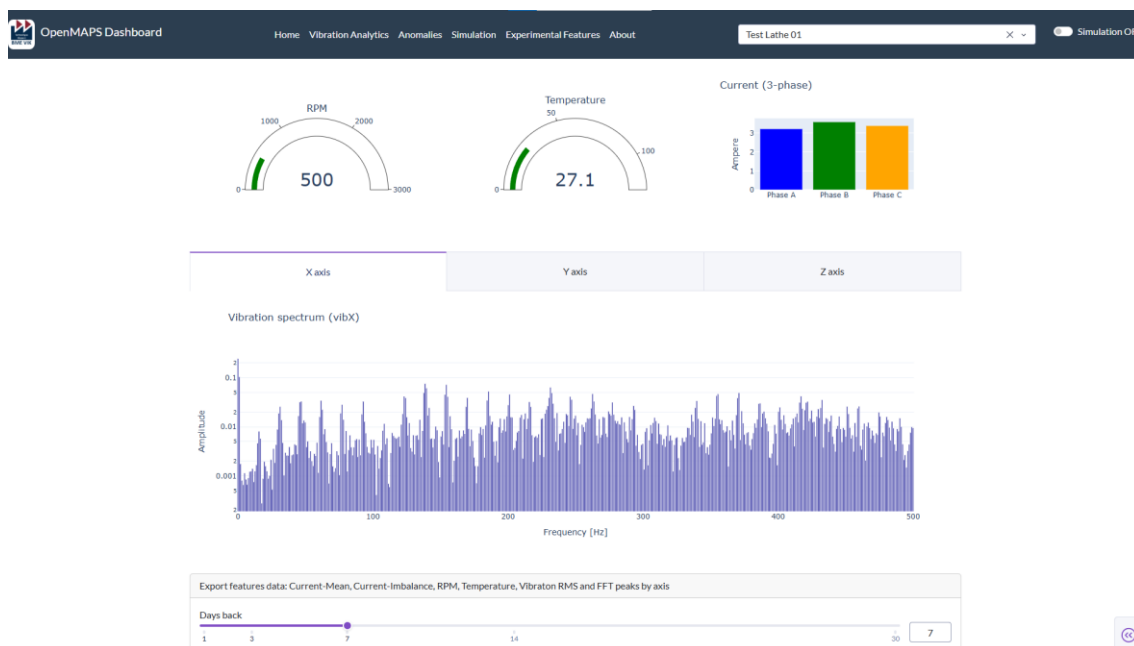
7.1.2. Dashboard főképernyő és aloldalak

7.1.2.1. Közös navigációs és gépválasztó sáv

Az oldal központi eleme a navbar, ami mindegyik oldal tetején helyezkedik el, egyben navigációs felület másrészt gépválasztó funkciót is betölt, valamint még a szimuláció is innen elindítható. Aloldalakra való navigáláshoz a *dash bootstrap components* könyvtár Navlink elemét használok fel, ami tematikusan rendezett aloldarakra navigál interakció után. Mellette található egy lenyíló menü, ahol a megfigyelni kívánt esztergagépet lehet kiválasztani, valamint tesztegységeket. Utóbbiak csak a valós ESP-k által kapott adatokat dolgozzák fel, míg előbbiek a szimulátor meghívásában szerepelnek. Továbbá megjelenítésre kerül még a projekt neve, „Compact-Maps” és a Technológiai Központ logója.

7.1.2.2. Főoldal

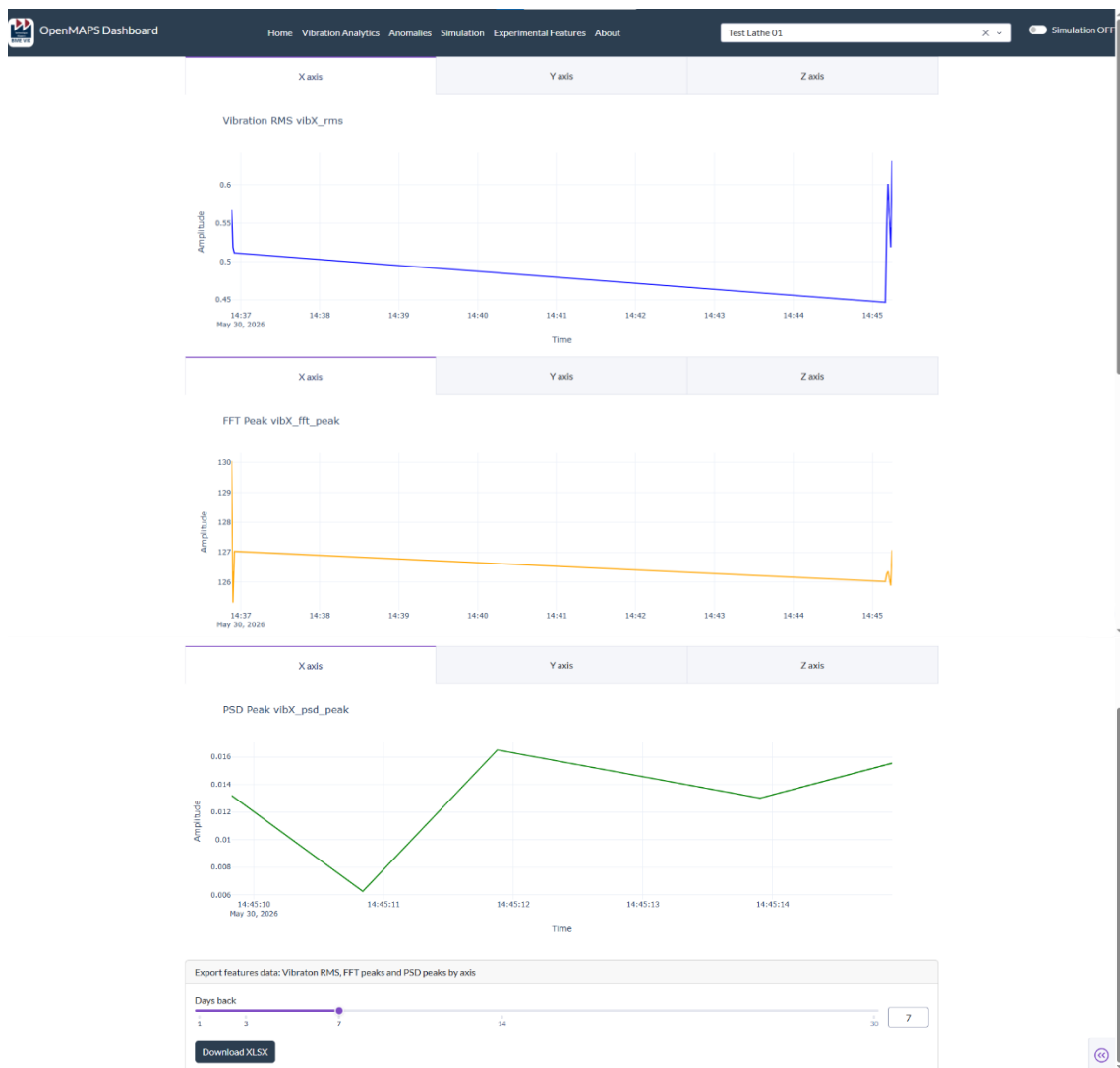
A *Home* oldal érdemileg nem változott elődjéhez képest, inkább dizájnban frissült. A gépkezelő felé továbbra is színes, animált diagrammok szolgáltatják az adatok megjelenítését. A főorsó fordulatszám és motorhőmérsékletéhez félkör alakú diagram lett kialakítva, míg a háromfázisú áramértékekhez fázisonkénti oszlopdiagram. A rezgésrétkékek reprezentálása sűrű oszlopdiagrammos formában jelenik, ahol a vízszintes tengely a 0-tól 500-ig terjedő frekvenciatartomány tartalmazza, a függőleges irány pedig a frekvenciaértékekhez tartozó amplitúdót. A diagram 3 aloldalt különböztet meg, ahol a tengelyenként felmerülő értékeket lehet vizualizálni. Illetve az oldal alján helyet kapott a főoldal adatait kimentő funkció, .xlsx formátumba, ami a csúszkán megadott napokig visszamenőleg a jelenlegi naptól kezdve érhető el. A főoldal kialakítása a 7.1.2.2 ábrán látható.



7.1.2.2 ábra – A Home főoldal és a szimulációs modullal feltöltött diagrammok

7.1.2.3. Több rezgésanalitikai módszer megjelenítése

A második oldal szerepe, hogy a több rezgésanalitikai módszert lehessen egyidőben elemezni és összehasonlítani az idő függvényén a feldolgozási módokat. Ehhez 3 különböző grafikont sorolok fel egymás után, kezdve az RMS (Root Mean Square) elemzéssel, ami a főoldalhoz hasonló módon 3 tengelyre bontva vetíti ki az adatokat egy szórásdiagramra. Ezután van az FFT csúcsokat ábrázoló, majd a PSD csúcsokat felsoroló diagram. Ezek az adatok legfeljebb 10 percre visszamenőleg jelennek meg a felületen, ami elég a trendelemzés vizsgálatára. Valamint az oldal alján itt is helyett kapott az adatok exportálási lehetősége, ami jelen esetben az aloldalon lévő elemzési módszerek minden tengelyre vonatkozóan. Az oldal elrendezése a [7.1.2.3-mas ábrán](#) látható.



7.2.3 ábra – Három elemzési módszer diagramja

7.1.2.4. Anomáliakövetés

A harmadik aloldal az „Anomalies” névre hallgat, ami lényegre törő metrikát publikál az idővonalon. Ehhez a tanulóalgorithmus által készített adatokat jeleníti meg szórásdiagramon kiemelve a ponthalmazokat 1 órára visszamenő intervallumban. Így látható a megmunkálások végén az eszköz természetes kopása lineáris görbén.



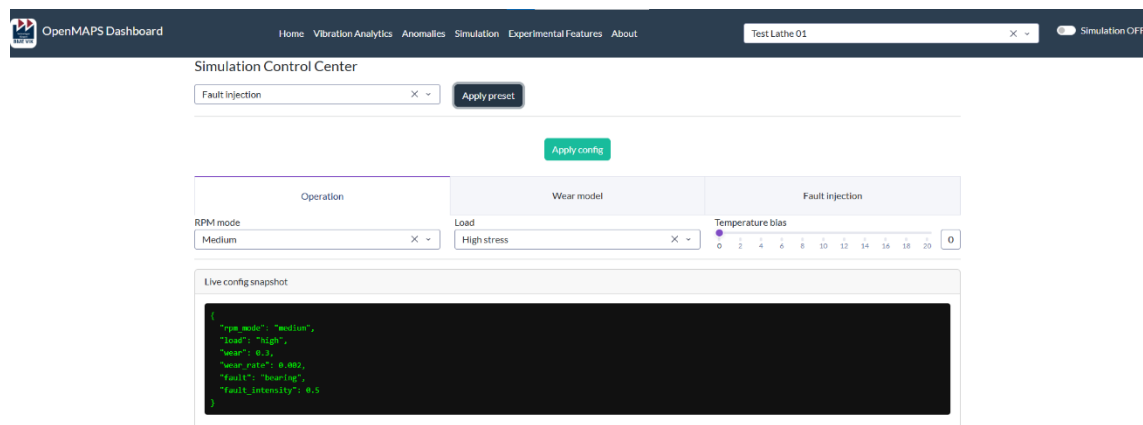
7.1.2.4 ábra – Megfigyelhető metrikaromlás ábrázolása az anomáliatrendek alapján

7.1.2.5. Szimulációs modul

A nyitóoldal mellett a második legfontosabb oldal a „Simulation”, ami a külön szimulációs modulnak a grafikus felületi implementálása. Itt történik az API megfelelő felparaméterezése, aminek design-ját a modern HMI felületek ihlették. Tartalmilag legfelül egy előre összeállított üzemválasztó található, ami előre definiált paraméterekkel látja el a végpontot. Az alábbi üzemmódokat építettem a rendszerbe, hogy több szituációval tudjam tesztelni főleg a tanulóalgoritmusok működését:

- Normal operation
- High wear test
- Fault injection
- Stress test

Üzemmódválasztó mellett az „Apply preset” gombbal lehet elmenteni a paraméterezést, ami a „Live config snapshot” mezőkön belül azonnal megjelenik. Itt olvasható formátumban jelennek meg a befolyásoló tényezők, így könnyen átláthatók az üzemállapotok. Előzetesen megalkotott üzemmódokon kívül lehetőség van tetszőleges állapot kiválasztására, valamint mélyebben beállíthatóvá válnak a hibainjektáló argumentumok is. Az egyedi üzemállapot mentéséhez az „Apply config” gomb megnyomásával ismét megjelennek a felparaméterezett mezők a „Live config snapshot”-ban. A szimulációs irányítópult a [7.1.2.5-ös képen](#) látható.



7.1.2.5 ábra – Szimulációs vezérlőpult

7.1.2.6. Tanulóalgorithmus kimenetek és metrikák

Az utolsó állapotokat megjelenítő oldal az „Experimental Features” névre hallgat, ahol főleg a tanulóalgorithmus működése közben keletkező metrikákat publikálom. Ennek az aloldalnak a funkciója kezdetben csak a RUL és „Health” számítás volt, de lényegesebbnek láttam főleg a log-okban megjelenő metrikák, állapot valószínűségek és még az ESP-k felől érkező állapotleírókat megjeleníteni.

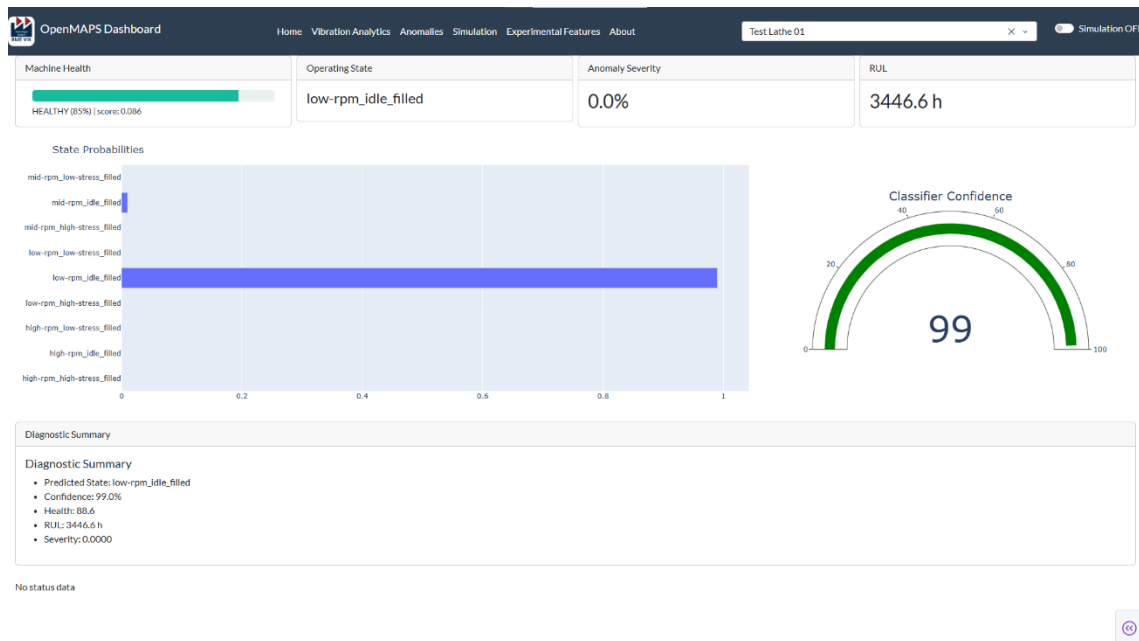
Felső sorban kártyás elrendezésben az alábbi értékeket jelenítem meg:

- Machine health: Progressbar, százalékos állapotinformáció és kalkulált metrika a tanulóalgorithmus által. A gépkezelő ezen információk alapján következtet az eszterga alapállapothoz képest az alkatrészek kopására és esedékes olajozásra.
- Operating state: A tanulóalgorithmus által kiválasztott állapot a 9 lehetségesből.
- Severity: Az eredeti állapothoz képest az elváltozás mértéke, minél nagyobb annál jelentős lehet a fizikai hiba.
- RUL: Becsléskalkuláció a fizikai gép hátralévő élettartamára órában.

Középső szekcióban két fontos metrikát ábrázolok:

- State probabilities: 9 vízszintes tengelyre orientált oszlopdiaagram, ahol az y tengelyen az egyes állapotok valószínűsége az éppen feldolgozott tényezők alapján legjobban megegyező üzemállapot látszódik. Ismert minták alapján egy állapot kimagaslóan valószínű (<0.90) a többi elhanyagolhatóan alacsony (0.01). Viszont, ha két állapot vagy több közel azonos értéket vesz fel, akkor következtethető hogy az eszköz romlás számottevő, mert egyértelműen egyik állapotba se sorolható.
- Classifier confidence: Az állapotválasztó algorithmus százalékos metrikája, ami a bizonyosságot fejezi ki az adott állapotba illeszkedéshez.

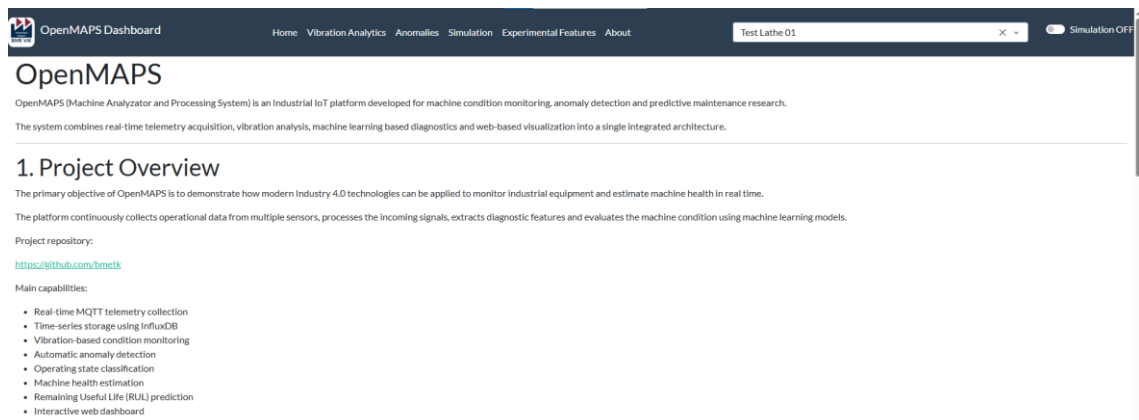
Legalul kártyás nézetbe felsorolásként jelennek meg az oldal metrikái formázatlanul, olvasható formátumban. Valamint, ha elérhetők az ESP-k felől diagnosztikai állapotadatok, akkor ezek több sorba „Badge”-eként állapotuk szerint szín kódolva jelennek meg. Az Experimental Features aloldal felépítése a [7.1.2.6-os ábrán](#) látható.



7.1.2.6 ábra- Kísérleti funkciókat összegző oldal

7.1.2.7. Projektről

Végző oldal az „About”, ahol kis bevezető után részletes leírást adok a projekt működéséről rétegekre bontva. Képletekkel mutatom be egyes adatok feldolgozásainak módját, amik érthetővé teszik a rendszer működését, kódrészletek bemutatása nélkül. Az *About* részlete a 7.1.2.7-es ábrán megtekinthető.



7.1.2.7 ábra – Projektösszefoglaló oldal részlete

8. Prediktív karbantartási funkciók

A rendszer másik nagyobb eleme a prediktív karbantartási modul, amelynek feladata a gép aktuális üzemállapotának meghatározása, az esetleges rendellenességek felismerése, valamint a berendezés „egészségi” állapotának és várható hátralévő élettartamának becslése. A megvalósítás a backend konténerrel egy modulban működik párhuzamosan az ottani utófeldolgozással, de induláskor egy előre definiált tanítóadatbázisra épül.

A rendszer beüzemelése utáni első lépésként betölti az összes rendelkezésre álló referenciaállapotot. Ezek az állapotok a különböző fordulatszám és terhelési kombinációkat reprezentálják, amelyek az adatgyűjtési szakasz során kerültek rögzítésre. A betöltés során minden állapothoz tartozó adathalmazból kiszámításra kerülnek a később felhasznált referenciaértékek, valamint inicializálódnak az anomáliadetektáló modellek. Az alkalmazás összesen kilenc normál üzemállapotot kezel, amelyek a kis, közepes és nagy fordulatszám, illetve az alapjárat, alacsony és magas terhelési szintek kombinációiból állnak elő.

A predikció során a rendszer először a beérkező mérési adatokból létrehozza az úgynevezett jellemzővektort (*feature vector*). Ebbe a vektorba kerülnek a villamos és mechanikai állapotot leíró mutatók, amik az áramfázisok átlagértéke, áramkiegyensúlyozatlanság mértéke, fordulatszám, hőmérséklet, valamint a rezgésjelből számított FFT csúcsértékek és RMS mutatók. Ezek a jellemzők jelentősen tömörebb formában reprezentálják a gép aktuális állapotát, mint a nyers szenzoradatok, így jobbak prognosztikai szempontból. Az elkészített jellemzővektor elsőként az állapotosztályozó modellhez kerül, aminek feladata annak meghatározása hogy a mérés melyik tanult üzemállapothoz hasonlít leginkább. A modell nem csupán egyetlen állapotot ad vissza, hanem minden ismert állapothoz egy valószínűségi értéket is rendel. Ezek az értékek azt mutatják meg, hogy az adott mérés milyen mértékben illeszkedik az egyes referenciaállapotokhoz. A legnagyobb valószínűséghez tartozó állapot kerül kiválasztásra aktuális üzemállapotként. A hozzá tartozó valószínűségi értéket a rendszer *confidence* mutatóként kezeli. Ez a jellemző fontos diagnosztikai információt hordoz, mivel magas érték esetén a modell nagy bizonyossággal tudja azonosítani az üzemállapotot, míg alacsony érték esetén a mérés több állapot határterületére esik.

A fejlesztés során bevezetésre került egy bizonytalansági küszöb is, amely alatt a rendszer figyelmeztetést generál, ilyenkor a modell ugyan képes állapotot választani, azonban annak megbízhatósága már nem tekinthető megfelelőnek.

Az állapotazonosítást követően az adott üzemállapothoz tartozó anomáliadetektáló modell kerül felhasználásra. Ez a modell már nem a teljes állapottérben vizsgálja a mintát, hanem kizárólag a kiválasztott referenciaállapoton belül értékeli annak eltérését. Az eredmény egy anomália pontszám (*anomaly score*), amely a normál működéstől való távolságot fejezi ki. Minél kisebb ez az érték, annál jelentősebb az eltérés a tanulási adathalmazban szereplő normál mintákhoz képest. Az anomália pontszámból további diagnosztikai mutatók kerülnek bevezetésre. A rendszer meghatározza az anomália súlyosságát (*severity*), amely normalizált formában mutatja meg az eltérés mértékét. Ezzel párhuzamosan kiszámításra kerül az „egészségi” állapotot reprezentáló (*health*) érték is, amely százalékos formában jelzi a berendezés állapotát. A magas egészségi érték normál működést, míg az alacsony érték fokozott elhasználódást vagy rendellenes működést jelez.

A prediktív karbantartás szempontjából kiemelten fontos a hátralévő hasznos élettartam (*Remaining Useful Life - RUL*) becslése. A rendszer ezt az anomália pontszám és az egészségi mutató alapján határozza meg. A számított érték nem abszolút élettartamot jelent, hanem a jelenlegi állapotból következő relatív becslést. A RUL mutató segítségével lehetőség nyílik a karbantartási beavatkozások előre tervezésére, valamint a meghibásodások kockázatának csökkentésére.

A teljes folyamat eredményeként a rendszer minden egyes mérési ciklus során meghatározza az aktuális üzemállapotot, annak bizonyosságát, az anomália mértékét, a gép egészségi állapotát, valamint a várható hátralévő élettartamot. Ezek az információk időbélyeggel ellátva kerülnek eltárolásra az InfluxDB adatbázisban, ahonnan a vizualizációs felület valós időben jeleníti meg őket a gépkezelő számára.

9. Tesztelés

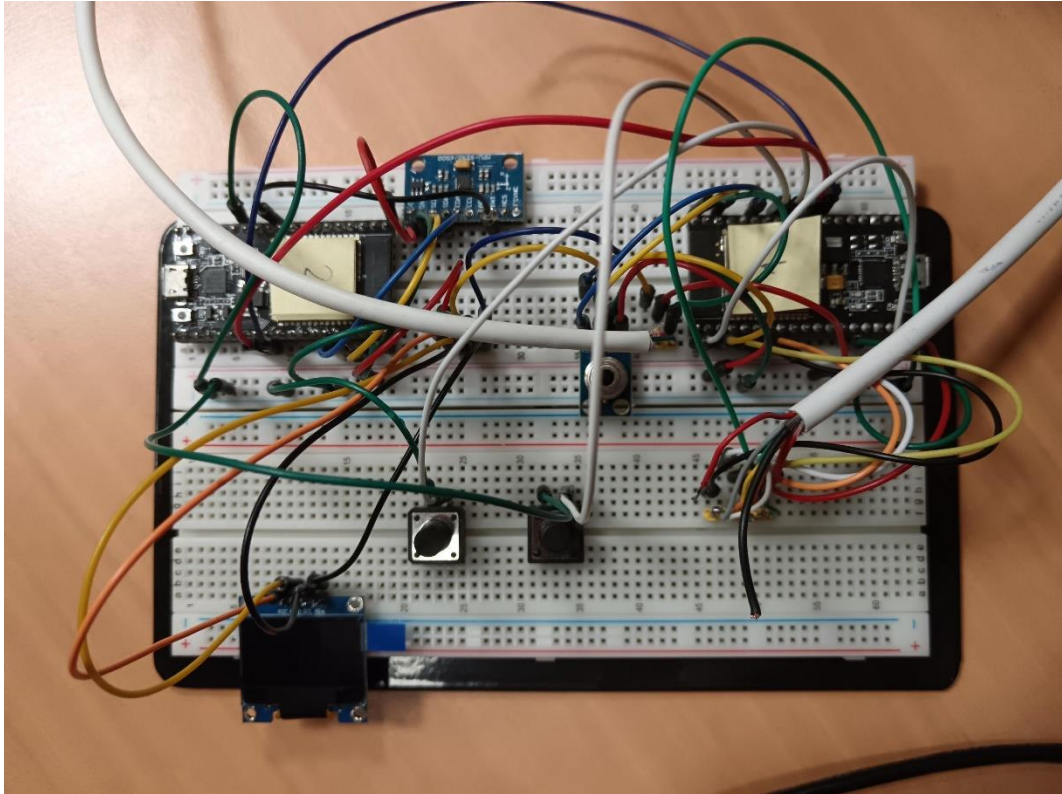
A félév megkezdésekor stabil elképzelésem volt a projekt fejlesztési iránya terén, extra, illetve meglévő funkciók kibővítésében. Kiindulási állapotnak egy stabil, működő rendszert vettem át, ami 2-2,5 éve folyamatosan működött és elmondások alapján a félév megkezdése előtt romolhatott el az esztergára kihelyezett modul.

Elvárás volt több adatgyűjtő rendszer építése, így legalább 3 szettnyi elektronikai alkatrész lett megrendelve az eredeti készítő listája alapján. Ez elhúzódó folyamat volt, úgyhogy a szenzoros fejlesztések főleg a 8. oktatási héttől kezdődtek, addig főleg a szerveroldali alkalmazáscsomag kiépítése zajlott.

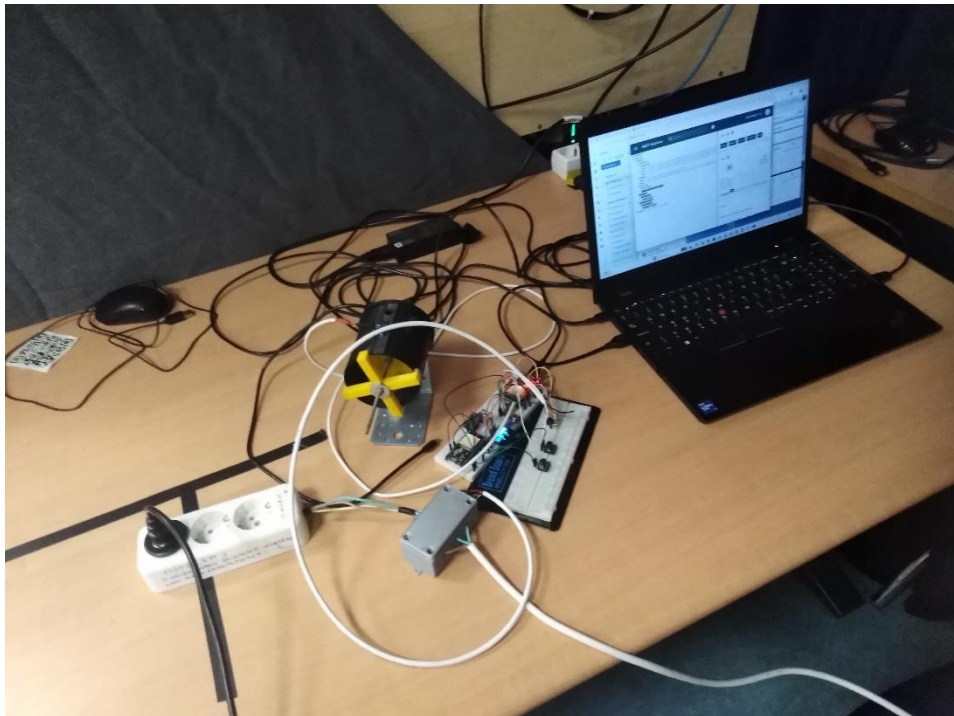
Adott voltak az előző projekt kódsorai, ezek főleg a vázát képeztek a fejlesztés korai szakaszának a gyors prototipizálás és rendszertesztelés jegyében. Az 5. oktatási hétre elkészült egy teljesen működő szerveroldali modul kezdetleges szimulációval és tanulóalgoritmussal. Rendszerteszteléshez főleg MQTT Explorer-t és Docker Desktop-ban lévő konténerek logjait tudtam használni.

Ezután sikerült előző projektekből hátramaradt két ESP32-öt helyreállítanom, hogy breadboard-ba illesztve 1-2 készleten lévő szenzorral el tudjam kezdeni a fejlesztést. Felleltünk talán az elődöm előtti projektből egy forgatható tengelyű hengeres dobót, amin felszerelve volt a réses IR érzékelő és a belső osztásos tárcsa egy vezetékre forrasztva. Ez tökéletes imitációja a valódi eszterga főorsójának és akár kézzel is lehet hajtani, de jelentősebben és főleg egyenletesen fúró segítségével. Miután a vezetékezése breadboard kompatibilis lett, jumper kábelekkel összekötöttem az ESP megfelelő portlábaival és elkezdtem a fordulatszám-mérés kialakítását.

A rendelések megérkezése után, a teljes mérőrendszerhez szükséges szenzorok mind elhelyezésre kerültek a breadboard-on. A kialakítás a 9/1 és 9/2-es ábrákon láthatóak.



9/1 ábra – Bal oldali ESP a rezgésmérésért, míg a jobb oldali a fordulatszám, hőmérséklet, áramfelvétel, gomb és megjelenítésért felel. Kettő között soros kapcsolat



9/2 ábra – Bal oldalon az elosztó vezetéke vannak átvezetve az árammérőn közepén meg a hengeres forgatható tengelyű orsó a fordulatoméréshez

10. Elért eredmények, tapasztalat

Kezdeti állapotához képest, az eredeti projektben elkészített mérőegység javítás után ismét teljesen működőképes állapotba került. Ezen felül sikerült egy újabb rendszert kihelyezni egy hallgatói esztergagépre, így megduplázva az adatgyűjtés lefedettségét valódi környezetben. Valamint egy bemutatható változatot breadboard és jumper kábelek segítségével.

Átdolgozásra került a szerveroldali szoftverkörnyezet is, elkülönül a több esztergától származó adat, így több berendezés monitorozására alkalmas egyidőben. Megvalósult egy megelőző karbantartást előrejelző tanulóalgoritmusokat felhasználó modul, ami a tanult minták alapján képes érzékelni a mechanikus szerkezetek kopását, kenőanyagok és gépszírok hiányát.

Kiegészült a webes felület fontos tanulóalgoritmus metrikákkal, RUL becsléssel és általános egészségállapot leírókkal, valamint külön idővonalon visszakövethető anomáliajelző funkcióval.

Továbbá a szimulációs modul teljesen újra lett építve, most valósághű adatokat publikál a feldolgozórendszer felé, amivel mélyebben lehet tesztelni a tanulóalgoritmusok működését, valamint jobb prezentációs célt is ellát.

Összességében a projekt stabil alapokra épült elődjére építve, kiegészítve több szinten az adatfeldolgozás tekintetében. A teljes scenárió működése hibatűrőbb és visszajelzőbb lett minden téren, valamint könnyen bővíthető és kódszinten optimalizált.

Teszteléshez főleg az eredeti tanári esztergát és onnan származó tanulóadatsort tudtam felhasználni modelltanuláshoz. Innen korlátozott mennyiségű főleg 7 perces működésekből 300 körüli minta keletkezett, ami általánosságban kevés a legtöbb modell számára (1.000 és 10.000 között már optimális). Viszont a kevés mintán kívül nagyobb probléma, hogy a betanult minták csak a tanári gépre vonatkoznak és azon belül is csak a felvett állapotokhoz hasonló üzemre. Tehát a hallgatói gépnek még azonos feltételek között is valószínűleg másabb rezgése van, mint a tanárinak ezért pontatlan lesz. Valamint a hallgatói gépet nagyobb fordulatszámú megmunkálásokhoz is használják, amire a tanulóhalmaz nincsen felkészítve. Így indokolt lett volna a hallgatói géphez saját tanulóhalmazt felvenni, de a félév végefelé erre már nem jutott idő.

11. Továbbfejlesztési javaslat

11.1.1. Kisebb mértékű fejlesztések

- 1 Ahogy az előző fejezet végén tárgyaltam, a hallgatói géphez külön tanulómintákat kéne rögzíteni, amihez a backend kódját szükséges kibővíteni, hogy a beérkező gépazonosító (lathe01 [tanári] vagy lathe02 [hallgatói]) alapján más tanulóadattal feltanított algoritmus vizsgálja. Tehát tanári géphez a tanári mintán tanult algoritmus dolgozzon, a hallgatóin pedig a hallgatói algoritmus. Ehhez főleg az esztergályosokkal kell egyeztetni, ami az anyaghasználat miatt költségekkel jár.
- 2 Tervem volt az internetfelöli forgalmat a szerverfelé korlátozni, de főleg validálni ezért az elérést HTTPS fölért biztosítani, valamint Domain név alapú elérést kialakítani, de erre idő hiányában nem került sor. Valamint olvastam TLS megoldásokat az MQTT üzenetek titkosítására is, ez is implementálható lehet.

11.1.2. Nagyobb mértékű fejlesztések

- 1 Jelenleg két esztergagépre van kihelyezve a szenzoros rendszer, ezek közül a tanári gépen ritkán dolgoznak, emiatt ott nem annyira hasznosítható a rendszer potenciálja a megelőző karbantartás terén. Így inkább egy új egységet kéne kiépíteni egy másik hallgatói esztergára vagy egy külső partnercéget érdemes felkutatni és ott kiépíteni a rendszert, mint terméket. Ehhez főleg kapcsolatok kellene, majd eszközfelmérés, szenzorépítés és végül kiépítés.
- 2 Szoftveres téren a jelenlegi adatelemzési módszereken kívül sok egyéb létezik még, amik beépíthetők lennének (Envelope, Modal, Joint time-frequency). Emellett egy Data-Science-el foglalkozó hallgató több tanulóalgoritmust is be tudna vezetni a működésbe.
- 3 Hardveres téren a teljes NYÁK-ot újra kéne dolgozni, több szenzort hozzáadni az állományba (például INMP441-M Omnidirekcionális mikrofon), jobban támogatni a csatlakozók időközi leszerelhetőségét és módszert találni a NYÁK elemeinek cserélhetőségére. Illetve az új NYÁK-hoz új 3D-s házat tervezni. Ehhez tapasztalt KiCad és SolidWorks-öt ismerő hallgató javasolt.

12. Irodalomjegyzék

- [1] Makiela, D. (2026. 04 12). Future Processing. Forrás: <https://www.future-processing.com/blog/data-collection-data-gathering-methods-benefits-and-best-practices/>
- [2] Tech Labs. (2026. 04 12). Forrás: <https://tech-labs.com/blog/evolution-industry-10-40-and-beyond>
- [3] CMC Global (2026. 4 12) Forrás: <https://cmcglobal.com.vn/digital-transformation/data-analytics-ai/evolution-of-big-data-learn-big-data-definition-and-how-it-evolved/>
- [4] HPE (2026. 04 12) HPE Glossary: Value of Data. Forrás: https://www.hpe.com/emea_europe/en/what-is/value-of-data.html
- [5] Kollektive (2026 05 19) Every Minute Matters Forrás: <https://www.kollektive.com/whitepapers/every-minute-matters>
- [6] OmiaScience (2026. 04 12) Predictive Maintenance Approaches: A Systematic Literature Review. Forrás: <https://www.jiem.org/index.php/jiem/article/download/8537/1131>
- [7] MDPI (2026.04 13) Predictive Maintenance and Intelligent Sensors in Smart Factory: Review. by Martin Pech, Jaroslav Vrchota and Jiří Bednář Forrás: <https://www.mdpi.com/1424-8220/21/4/1470>
- [8] InfluxData (2026. 04 13) Sensor Data: What Is It & How to Use It?. Forrás: <https://www.influxdata.com/sensor-data-is-time-series-data/>
- [9] Github (2026. 04 26) Kereszty Márk szakdolgozatának publikus Github linkje: <https://github.com/bmetk/IoTDataCollection>
- [10] Diplomaterv (2026. 05 11) Kereszty Márk szakdolgozata Forrás: <https://diplomaterv.vik.bme.hu/hu/Theses/IT-alapu-adatgyujto-adatelemzo-rendszer/Attachment/75077>
- [11] Espressif (2026. 05 20) Free RTOS docs Forrás: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/freertos_idf.html?utm_source=chatgpt.com#critical-sections

13. Függelék

13.1. I. Függelék Nyilatkozat generatív mesterséges intelligencia alkalmazásáról

- ☐ **Nem használtam** semmilyen generatív MI segédeszközt.
- ☒ **Használtam** generatív MI segédeszközt. Az MI-vel generált tartalmakat ellenőriztem, a generált kimenetek valóságtartalmáról meggyőződtem, az alábbi táblázatban megfelelően jelöltem minden használatot.

Felhasználási módok	Generatív MI eszköz(ök) neve	Érintett részek (fejezet, oldalszám, hivatkozás)	Használat becsült aránya (felhasználási módonként)
Irodalomkutatás	ChatGPT	22-23. oldal	30%
Prompt lényegi része	„Mutass be pár népszerű ipari IoT szoftver platformot, mint például az AWS IOT.”		
Programkód generálása	ChatGPT	63-64. oldal	30%
Prompt lényegi része	„Készítsd el a State Classifier implementációját, ami a RandomForestClassifier modell segítségével a feltanult minták alapján kiválasztja a bemeneti értékek alapján a legvalószínűbb üzemállapotot többségi szavazás módszerrel.”		
Új ötletek, megoldási javaslatok generálása	-	-	-
Prompt lényegi része	-		
Vázlat létrehozása (szövegstruktúra, vázlatpontok)	-	-	-
Prompt lényegi része	-		
Szövegblokkok létrehozása	-	-	-
Prompt lényegi része	-		

Képek generálása illusztrációs célból	-	-	-
Prompt lényegi része	-		
Adatvizualizáció, grafikonok generálása adatpontok alapján	-	-	-
Prompt lényegi része	-		
Prezentáció készítése	-	-	-
Prompt lényegi része	-		
Egyéb (nevezze meg)	Copilot	-	5%
Prompt lényegi része	Főbb függvények és kódrészek kommentezése		
Összesített százalékos érték (a feladat érdemi részére nézve)			5%
Összesített érték rövid, szöveges indoklása: A nagynyelvi modellek által készített tartalmak a szakdolgozat feladatainak érdemi részének kisebb hányadát teszik ki. Új és régi felhőalapú IoT platformok keresésére használtam rávezető információkkal. A modellválasztó State Classifier kód nagyrészét ChatGPT generálta, ami 100 sor körül van. Összes kódsort tekintve azonban ez elenyésző mennyiség. Főbb ESP funkciókhoz, függvényekhez és kódrészekhez, valamint Python adatfeldolgozó függvényekhez és tanulóalgoritmus deklarációkhoz és felhasználói felület minden oldalához kisebb ismertető leírásokat készített a Copilot. Ezzel áttekinthetőbbek lettek az egyes funkciók működései főleg fejlesztőknek.			